Yoobee College of Creative Innovation
Master of Software Engineering — Level 8

MSE802 Quantum Computing

Assessment 2 — Quantum Project (60%)

Designing, Running and Evaluating Quantum Circuits:

Bell States, Qiskit and QASM Circuits, Quantum Tic-Tac-Toe and a Quantum Classifier on the Quokka Device

GROUP PROJECT

Student 1	Anish Shrestha — Student ID 270889991
Student 2	Sandhya Tiwari — Student ID 270918511
Lecturer	Dr Helena Bahrami
Course	MSE802 Quantum Computing (15 credits)
Date	4 November 2026
GitHub repository	github.com/anishnz/quantumcomputing_YB_Anish
Referencing style	APA 7th edition

Auckland, New Zealand

Declaration of Originality

We, the undersigned members of this group, declare that this report and the accompanying code are our own group work, completed for MSE802 Assessment 2. All sources of information, code and ideas from others, including the lecture notes and the supplied lab notebooks, have been acknowledged and referenced in APA 7th style. This work has not been submitted for any other course. We understand Yoobee College's policies on plagiarism and collusion as described in the Student Handbook.

We confirm that the results reported here come from the notebooks submitted with this report. Results produced on a local simulator because the Quokka device was unavailable are clearly labelled as such and are not presented as device measurements.

Use of digital tools: Python libraries (Cirq, Qiskit, NumPy, Matplotlib) were used for all computation. A generative AI assistant was used to support code drafting, debugging and language editing; all code was run, checked and understood by the group, and the analysis and conclusions are our own. [Edit this sentence to match Yoobee's AI-use policy and your actual use.]

Name	Student ID	Signature / date
Anish Shrestha	270889991	
Sandhya Tiwari	270918511	

Table of Contents

Declaration of Originality.... 1
Table of Contents.... 2
Abstract.... 4
1. Introduction.... 5
1.1 What is this project about?.... 5
1.2 Which learning outcomes does each task show?.... 5
1.3 Which tools and device did we use?.... 5
1.4 How should the reader read the results?.... 6
2. Project Setup and How to Run.... 7
2.1 What is in the project folder?.... 7
2.2 How do we run it on a Windows PC?.... 7
2.3 How do we run it on a Mac?.... 8
2.4 Where can we see the output?.... 8
2.5 How do we run it in Google Colab?.... 8
2.6 What happened with the Quokka device, and which devices work?.... 9
2.7 What alternative code did we provide?.... 11
3. Task 1 – Entanglement Demonstration (Cirq + Quokka).... 12
3.1 What is a Bell state, and why is it special?.... 12
3.2 Which gates make the state, and why?.... 12
3.3 How did we build it in Cirq?.... 12
3.4 What did the measurement show?.... 13
3.5 Why is the split not exactly 500/500?.... 14
3.6 Would a real superconducting or ion-trap computer give the same result?.... 14
3.7 What does Task 1 prove?.... 14
4. Task 2 – Qiskit Circuits and a Custom QASM Circuit.... 15
4.1 Introduction: what are qubits and quantum circuits?.... 15
4.2 Circuit 1: Why did we choose these gates for block C?.... 15
4.3 Circuit 1: What QASM code was sent to Quokka?.... 16
4.4 Circuit 1: What happens to the qubits, step by step?.... 16
4.5 Circuit 1: What did Quokka return, and what do the results mean?.... 17
4.6 Circuit 2: Why did we design a quantum half adder?.... 17
4.7 Circuit 2: What is the QASM code?.... 18
4.8 Circuit 2: What does the theory predict?.... 18
4.9 Circuit 2: Can Quokka run every gate in our design?.... 18
4.10 Circuit 2: What did Quokka return?.... 19
4.11 Analysis: How do the two circuits compare?.... 19
4.12 Analysis: What do superposition and entanglement look like in the data?.... 20
4.13 Conclusion: What did Task 2 teach us?.... 20
5. Task 3 – Investigating a Quantum Code: Quantum Tic-Tac-Toe.... 21
5.1 What is the purpose of the program?.... 21
5.2 How is the code organised?.... 21
5.3 What was missing, and what did we complete or fix?.... 21
5.4 How does a move become a gate? (Circuit generation).... 22
5.5 How does a rotation move change the odds?.... 22
5.6 What happened when we played? (Four recorded games).... 23
5.7 What do the games show? (Analysis).... 24
5.8 Did we also play the game by clicking the buttons?.... 25
5.9 Can the game circuit run on Quokka?.... 26
5.10 Which quantum gates does the game use? (Gate section).... 26
5.11 How do these gates fit the wider theory of quantum logic gates?.... 27

5.12 What are the strengths and limits of the program? 27
6. Task 4 – Machine Learning Quantum Analysis 28
6.1 What problem does the code solve? (Problem formulation) 28
6.2 Which part of the code is the input to the circuit? 28
6.3 How does the circuit work? (Quantum part, excluding the optimiser) 28
6.4 What did we add to monitor every iteration? 29
6.5 What do the two required graphs show? 30
6.6 How did we make the code run without the quantum computer or circuit? 31
6.7 What were the differences in efficiency and effectiveness? 31
6.8 Did anything not work as expected? 33
6.9 What is our overall judgement for Task 4? 33
7. Our Computer versus Quokka: Performance and Outcome 34
7.1 How did we compare them fairly? 34
7.2 Which is faster for our circuits? 34
7.3 Do they give the same outcome? 35
7.4 When would Quokka become the better choice? 36
7.5 What about a full workload such as Task 4? 36
7.6 What do we conclude? 37
8. Critical Discussion Across the Four Tasks 38
8.1 How does the evidence meet each learning outcome? 38
8.2 What are the main limitations of our work? 38
8.3 What would we do next? 38
9. Development Process and Code Walkthrough 39
9.1 What process did we follow? 39
9.2 How does the shared helper work? 39
9.3 What does each notebook cell do? 41
9.4 What went wrong during development, and how did we fix it? 42
9.5 How did the code change over time? (Version history) 42
10. Conclusion 44
References 45
Appendix A – Submission Structure and How to Run 47
Appendix B – Key Code Excerpts 47
Appendix C – Evidence Screenshots 49
C.1 Quokka website and device status (4 November 2026) 49
C.2 Task 3 games played by clicking in Jupyter Notebook 51
C.3 Notebooks executed on the Quokka device 53

(If page numbers are missing, right-click the table and choose Update Field.)

Abstract

This group project designs, runs and evaluates quantum circuits for the four tasks of MSE802 Assessment 2. Every circuit was written in Python (Cirq or Qiskit) or directly in OpenQASM 2.0, executed with **1000 shots**, and run on the **Quokka device** (quokka2) on 4 November 2026. In **Task 1**, a Hadamard gate and a CNOT gate prepare the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$; all 1000 shots on Quokka were correlated (501 × 00, 499 × 11). In **Task 2**, the given H and X gates are followed by our block C (CNOT, Z, CNOT, H), which creates a Bell pair, changes its phase and reads it with a Bell measurement. The pair was shown to be entangled (entropy 1 bit; always-opposite bits in a checkpoint run), and the full circuit gave 11 in 1000 of 1000 shots, while a control without Z gave `10` in 1000 of 1000. Our own four-qubit QASM circuit, a **quantum half adder**, computed all four one-bit additions in a single circuit with 1000 correct results out of 1000. In **Task 3**, we completed the quantum tic-tac-toe game (four missing gates, the eight winning lines and five fixes), analysed four recorded games, played three more by clicking, and ran a game circuit on Quokka. In **Task 4**, we analysed a tree-shaped variational classifier, logged the output and time of all 100 training iterations (900 circuits on the Quokka device (quokka2)), and compared it with three local versions. The circuit classified all four images. The exact local simulation was about **3490 times faster**, while linear classical models failed because the data is an XOR-type problem. As the lecturer requested, we also **compared our own computer with four Quokka devices**. Outcomes were identical within shot noise. Our computer was 12–16 times faster for small circuits, because more than 99% of Quokka's time is network delay, but its cost grows as 2^n, and at about 24 qubits Quokka becomes as fast. Sources are cited in APA 7th style.

Keywords: Bell state, entanglement, Bell measurement, Cirq, Qiskit, OpenQASM, Quokka, half adder, quantum tic-tac-toe, variational quantum classifier, SPSA, performance comparison

1. Introduction

1.1 What is this project about?

The aim of Assessment 2 is to **implement different quantum circuits, test programs and algorithms on a practical quantum computer, and judge how effective these systems are**. The four tasks move from a two-qubit circuit (Task 1) to a hand-written circuit (Task 2), to reading someone else's quantum code (Task 3), and finally to a small quantum machine-learning model (Task 4). Each task is answered with working code, real measurements, an expected-versus-measured comparison and a short critical discussion. The objective has three parts, and this report answers each one with evidence: **diverse circuits** are designed in Tasks 1–4 (Sections 3–6); they are **tested on a practical quantum computer**, because every task ran on the Quokka device; and the **effectiveness of the quantum system** is judged in Section 7, which compares our own computer with four Quokka devices for outcome and performance. Section 8.1 maps the evidence to the learning outcomes (LO1, LO2, LO4).

1.2 Which learning outcomes does each task show?

Table 1. *Assessment tasks, weights and learning outcomes*

Task	Topic	Weight	LOs	Main evidence in this report
Task 1	Entanglement demonstration (Cirq)	25%	LO2	Bell circuit, state maths, 1000-shot results, statistics
Task 2	Qiskit circuit + own QASM circuit	35%	LO1, LO2	Gate-by-gate maths, two circuits, theory vs measurement
Task 3	Investigate quantum tic-tac-toe	25%	LO1, LO2, LO4	Completed code, four recorded games, gate section
Task 4	Machine-learning quantum analysis	15%	LO1, LO2, LO4	Circuit analysis, iteration log, two graphs, local comparison

1.3 Which tools and device did we use?

All code runs in Jupyter notebooks (VS Code or Jupyter) with Python 3.12 or newer. Circuits are built with **Cirq** 1.7 (Task 1; Cirq Developers, 2026) and **Qiskit** 2.6 (Tasks 2–4; Javadi-Abhari et al., 2024) and sent to the device as **OpenQASM 2.0** text (Cross et al., 2017). The device is **Quokka**, a personal quantum computer developed at the University of Technology Sydney. Quokka is an **emulator**: classical hardware that reproduces the behaviour of an ideal, fault-tolerant 30-qubit quantum computer (University of Technology Sydney [UTS], 2024). The course notebook and the official Quokka notebooks reach it with an HTTP request containing a QASM script and a shot count, `POST /qsim/qasm` with `{"script": ..., "count": 1000}` (Bahrami, 2026d; Quokka Computing, n.d.-a). The Quokka docs list the gates it accepts (h, x, y, z, rx, ry, rz, cx, swap, cphase, U), so our helper also rewrites any other gate into this set before sending (Quokka Computing, n.d.-b). Quokka Computing runs several cloud devices (quokka1 to quokka11); our helper tries them in order and records which one answered. This matters for the analysis: because Quokka emulates an *error-free* machine, its results should match ideal theory except for random shot noise, unlike today's noisy hardware (Preskill, 2018).

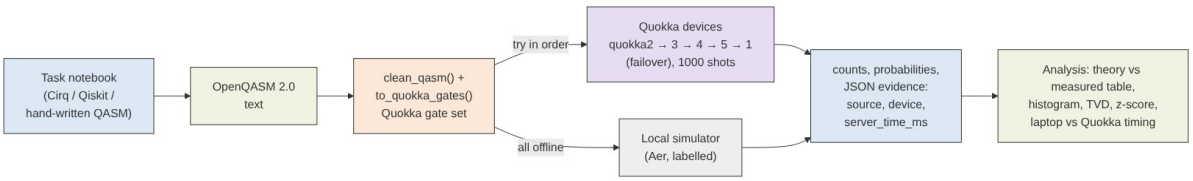


Figure 1. *Shared execution pipeline used by Tasks 1, 2 and 4 (file common/quokka_helper.py)*

To avoid repeating code, we wrote one shared helper (Figure 1). It (1) cleans the QASM text, removing the `include "qelib1.inc"` line, comments and barriers that Quokka does not accept; (2) posts it to the first online Quokka in its device list with 1000 shots; (3) turns the raw shot list into counts; and (4) saves every run as JSON evidence with its source, device name, time stamp, total run time and the device's own processing time.

> Note on the Quokka device. From 30 to 31 October 2026 the course address quokka1 was offline (ERR_NGROK_3200), so the first draft used a labelled local simulator. On 4 November 2026 four cloud Quokkas (quokka2–5) were online. Every task was re-run with 1000 shots, and all results in this report come from the **Quokka device quokka2**. Each JSON evidence file stores its source (QUOKKA DEVICE or LOCAL SIMULATOR), the device name and the device's own processing time. Section 7 compares our own computer with Quokka, as the lecturer asked.

1.4 How should the reader read the results?

Bit-strings follow the Qiskit convention: the **left-most character is the highest-numbered bit**. Two measures are used to compare theory with measurement. The **total variation distance (TVD)** is half the sum of the absolute differences between two probability distributions: 0 means identical and 1 means completely different. For a 50/50 outcome, the **z-score** tells us whether a count is within normal shot noise: with 1000 shots the standard deviation is $\sqrt{(1000 \times 0.5 \times 0.5)} \approx 15.8$, so counts between about 468 and 532 are normal.

2. Project Setup and How to Run

All code, results and this report are stored in the GitHub repository **https://github.com/anishnz/quantumcomputing_YB_Anish** in the folder `Assignment_2_Quantum`. The project can be run in four ways: one click on Windows, one command on any computer, cell by cell in VS Code or Jupyter, or fully online in **Google Colab** with no installation. Every route runs the same notebooks with **1000 shots** per circuit and saves the same evidence files.

2.1 What is in the project folder?

Table 2. *Project folders and files (one folder per task, as the brief requires)*

Folder / file	What it contains
Task_1_Bell_State/	Task 1 notebook (Cirq Bell state) and results/ (counts JSON, table CSV, histogram, QASM)
Task_2_Qiskit_QASM/	Task 2 notebook, our custom_half_adder.qasm, results/ (circuits, counts, histograms)
Task_3_Quantum_TicTacToe/	completed game notebook, the original supplied notebook, results/ (game circuits, summaries)
Task_4_Quantum_ML/	modified ML notebook (.ipynb, as required), the original notebook, results/ (iteration logs, graphs)
Alternative_Code/	a second way to solve each task: Qiskit Bell state, Cirq circuits, terminal game, Statevector classifier
common/quokka_helper.py	shared code: clean QASM, send to Quokka, labelled fallback, counts, charts
Device_Comparison/	laptop vs Quokka comparison script, results (JSON, CSV, charts) and dated Quokka screenshots
run_project.py	the runner: menu, all tasks together, one task, alternative code, Quokka status, laptop vs Quokka comparison
run_project.bat / run_all.bat	Windows double-click launchers
run_project.command / run_all.command / run_project.sh	macOS double-click launchers (and a Linux/macOS terminal script)
Colab_Run_All_Tasks.ipynb	runs the whole project in Google Colab
requirements.txt, MAINTENANCE.md, README.md	package versions, maintenance guide, step-by-step instructions
Report/	this report (Assignment2.docx and Assignment2.pdf) and its figures

2.2 How do we run it on a Windows PC?

1. Install **Python 3.12 or newer** (tick "Add to PATH") and download the repository: green **Code** button → **Download ZIP**, or `git clone https://github.com/anishnz/quantumcomputing_YB_Anish.git`.
2. Open the `Assignment_2_Quantum` folder and **double-click `run_project.bat`**. The first time, it creates a virtual environment and installs `requirements.txt` (a few minutes).
3. A menu appears: **1–4** runs one task, **A** runs Tasks 1–4 together, **L** runs the alternative code, **S** tests every Quokka device, **C** compares this computer with Quokka. `run_all.bat` runs all four tasks without the menu.

4. From a terminal the same runner takes arguments: `python run_project.py all`, `python run_project.py 2`, `python run_project.py status` or `python run_project.py compare`.

2.3 How do we run it on a Mac?

1. Install **Python 3.12 or newer** from python.org (or `brew install python`) and download the repository as above.
2. In Finder, open `Assignment_2_Quantum` and **double-click `run_project.command`** (menu) or **`run_all.command`** (Tasks 1–4 together). Terminal opens, creates `.venv` and installs the packages the first time, then runs exactly the same runner as on Windows.
3. If macOS says the file cannot be opened because it is from an unidentified developer, right-click it and choose **Open → Open** once. If it is not executable after unzipping, run `chmod +x run_project.command run_all.command` in Terminal.

```
python run_project.py all   (4 November 2026, Tasks 1-3; Task 4 re-run after the online-check fix)

Quokka devices (tried in order quokka2, quokka3, quokka4, quokka5, quokka1): online: quokka2, quokka3, quokka4, quokka
RUNNING Task 1 - Bell state (Cirq)
  finished in 8.3 s
  RESULTS:
    source: QUOKKA DEVICE (quokka2)
    counts (1000 shots): {'00': 501, '11': 499}   expected: only 00 and 11, ~500 each
RUNNING Task 2 - Qiskit circuit + custom QASM
  finished in 10.5 s
    2A checkpoint H,X,CNOT,Z      : {'01': 532, '10': 468}   expected: only 01 and 10
    2A full H,X,CNOT,Z,CNOT,H     : {'11': 1000}   expected: only 11
    2A control without Z          : {'10': 1000}   expected: only 10
    2B half adder counts: {'0000': 261, '0111': 242, '1001': 250, '1010': 247}   expected: 0000, 1001, 1010, 0111 (~25
RUNNING Task 3 - Quantum tic-tac-toe
  finished in 5.2 s
    game1_no_moves   X wins  392/1000   O wins  405/1000
    game2_diagonals  X wins  705/1000   O wins  210/1000
    game3_not_gate   X wins  135/1000   O wins  694/1000
    game4_swap       X wins  404/1000   O wins  289/1000
RUNNING Task 4 - Quantum machine learning
  finished in 631.6 s
    Quantum circuit (Quokka)                    4.0487 s/iter   final loss    0.0   all 4 images 1.0
    Local-A: same circuit, exact NumPy         0.00116 s/iter   final loss 0.0003   all 4 images 1.0
    Local-B: classical logistic regression       5e-05 s/iter   final loss 0.2961   all 4 images 0.75
    Local-C: classical + pairwise features     0.00019 s/iter   final loss 0.2057   all 4 images 0.75
    plots: Task_4_Quantum_ML/results/task4_iteration_vs_output.png and ..._vs_time.png

ALL FOUR TASKS FINISHED. Open outputs_html/ to see every result.
```

Figure 2. *Terminal output of python run_project.py all on 4 November 2026 (shortened): Quokka device status, each task's run and its key results*

2.4 Where can we see the output?

- **In the terminal:** after each task, the runner prints the key numbers (Figure 2).
- **Inside each notebook:** circuits, tables, histograms and graphs are saved in the executed `.ipynb`, ready to open in VS Code, Jupyter or Colab.
- **As web pages:** `outputs_html/` contains one HTML file per task; double-click to open in any browser.
- **As evidence files:** each `results/` folder stores the raw counts with their source (QUOKKA DEVICE or LOCAL SIMULATOR), CSV tables and PNG charts used in this report.

2.5 How do we run it in Google Colab?

Colab runs the notebooks on Google's computers, so nothing needs to be installed. The repository is connected to Colab through its GitHub address:

1. Open **https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Colab_Run_All_Tasks.ipynb** (or, in Colab, choose **File →**

Open notebook → GitHub, enter `anishnz/quantumcomputing_YB_Anish` and pick `Colab_Run_All_Tasks.ipynb`).

2. Click **Runtime → Run all**. The notebook clones the repository, installs Cirq and Qiskit, tests Quokka, runs Tasks 1–4 together, shows the results and charts, runs the alternative code, and offers a zip of all outputs.
3. To run one task, open its notebook from GitHub the same way. Its first code cell detects Colab, downloads the project and moves into the task folder, so the rest runs unchanged.

Colab test run (31 October 2026). The notebook was saved to Google Drive as a Colab project named `Assignment2_MSE802_Quantum_Project.ipynb` and run with **Runtime → Run all**. All four tasks finished without errors (Figure 3). Tasks 1–3 gave exactly the same counts as on the PC, because the local simulator uses a fixed seed. Task 4 took 160 s instead of about 90 s (0.99 s instead of about 0.6 s per iteration), because Colab's free CPU is slower than the PC; the loss curve and accuracy were identical. A first attempt showed one portability problem: the notebooks were executed by a Jupyter kernel without Cirq installed. We fixed it by making `run_project.py` install any missing packages into its own Python and register a matching kernel.

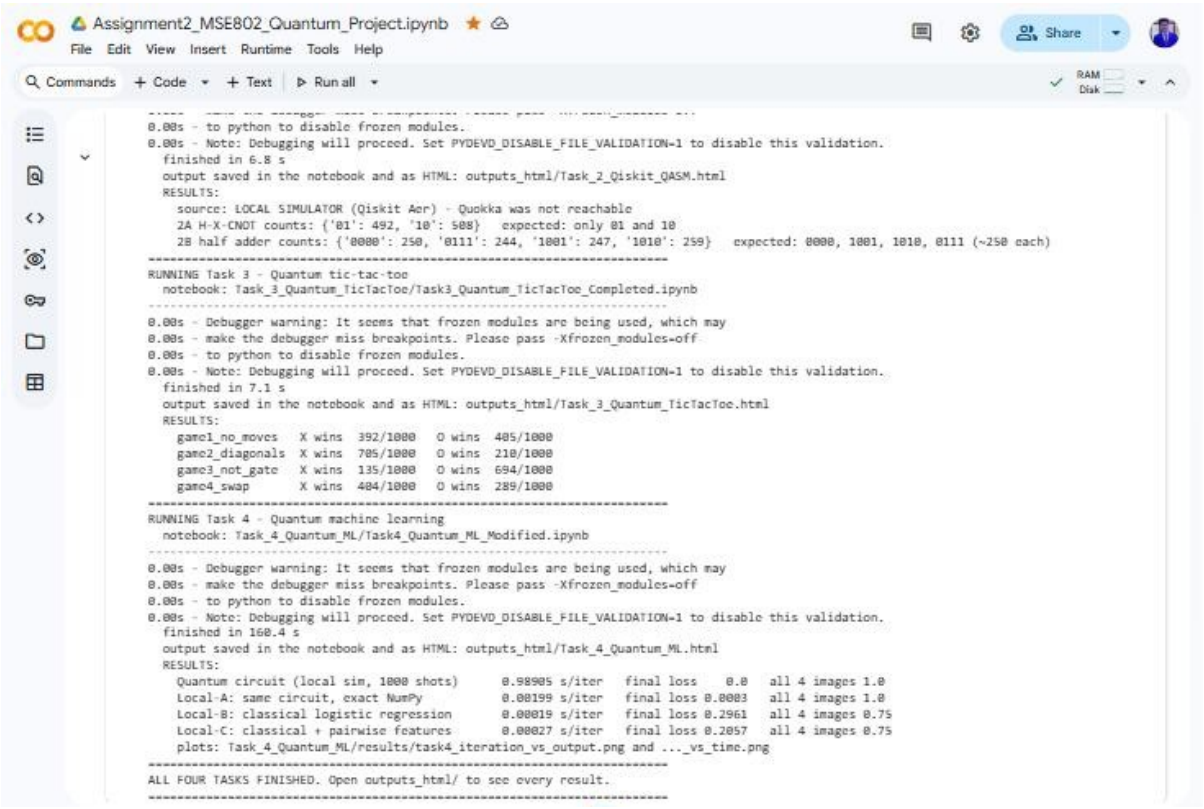


Figure 3. *Tasks 1–4 run together in Google Colab (notebook Assignment2_MSE802_Quantum_Project.ipynb, Quokka offline so labelled LOCAL SIMULATOR)*

2.6 What happened with the Quokka device, and which devices work?

Quokka has two parts that are easy to confuse. The **Quokka notebooks** (Start Here, Docs, the lab notebooks) are public Google Colab pages with instructions. The **Quokka simulators** are separate servers, reached at `http://<name>.quokkacomputing.com/qsim/qasm` through a tunnelling service called ngrok (Quokka Computing, n.d.-a, n.d.-b). A server must be switched on by its owner. When it is off, the address replies **HTTP 404 with the code ERR_NGROK_3200**, which means the address exists but no server is connected behind it. No login or API key is involved: the official notebooks send no key or token, only the device name.

Table 3. *Quokka connection tests, 30 October to 4 November 2026*

Date (NZ time)	Test	Result
30–31 Oct	POST /qsim/qasm to quokka1 from the project workspace, Google Colab and a web browser	HTTP 404, ERR_NGROK_3200 (offline)
30–31 Oct	quokka2, quokka3, quokka4 (http and https)	all ERR_NGROK_3200 at that time
31 Oct	automatic re-check every 2 minutes for 2 hours	offline the whole time
4 Nov, 17:57	1000-shot-style Bell circuit (10 shots) to quokka1–quokka20	quokka2, 3, 4, 5 returned correct results; all others 404
4 Nov, 18:01	official Fleet Status board on quokkacomputing.com/get-started (signed in), 11 nodes, GET /qsim/ping	4 online (quokka2–5), 7 offline (quokka1, quokka6–11)
4 Nov, 18:14–18:31	all four tasks re-run with run_project.py	Tasks 1–4 answered by quokka2 (Task 4: 900 circuits)

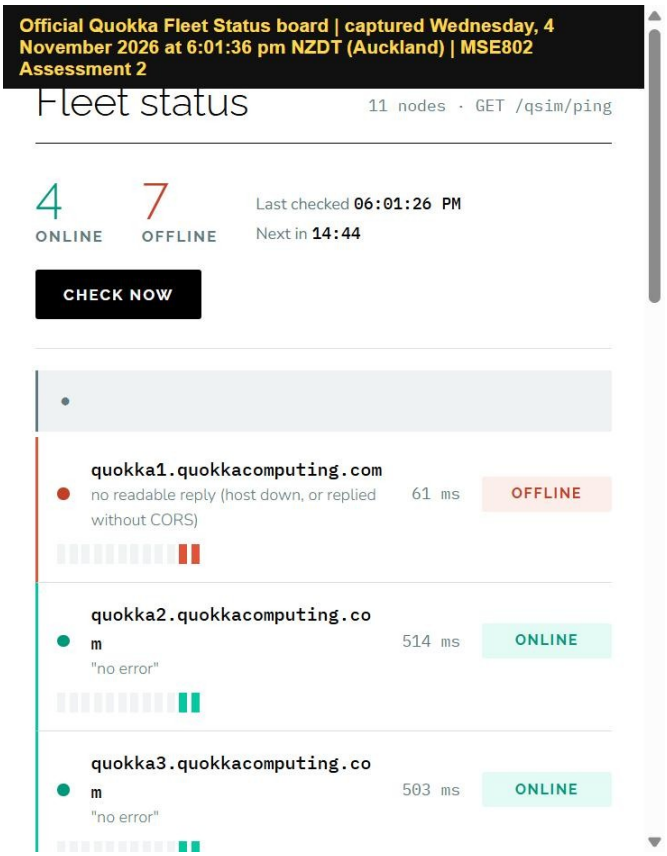


Figure 4. *Official Quokka Fleet Status board, captured 4 November 2026 at 6:01 pm NZDT (full screenshots in Appendix C)*

Three conclusions follow. (1) The outage was not caused by our code. The same request format later worked unchanged on quokka2–5. (2) A single hard-coded device is a weak design. The helper now keeps an ordered **device list** (`quokka2, quokka3, quokka4, quokka5, quokka1` in `quokka_config.json`). It moves to the next device if one fails, and records in every result which device answered and how long the device itself worked (`server_time_ms`). (3) The labelled fallback kept the project honest during the outage. The first draft's simulator results were never presented as

device data, and one command replaced them with Quokka results when the devices returned. All results in this report now come from **Quokka (quokka2)**, unless a table says otherwise.

2.7 What alternative code did we provide?

Table 4. *Alternative implementations in Alternative_Code/ (run with python run_project.py alt)*

Task	Alternative	Why it is useful
Task 1	Bell state written in Qiskit instead of Cirq	shows the result does not depend on the library; same QASM reaches Quokka
Task 2	Both circuits written in Cirq and run on Cirq's own simulator	independent check of Part A and the half adder
Task 3	Text-only tic-tac-toe in the terminal (X n, O n, NOT n, SWAP a b, PROB, MEASURE)	plays without widgets; --demo replays a game
Task 4	Same classifier as a Qiskit circuit, exact Statevector output, COBYLA optimiser	shows the circuit, not the optimiser, solves the problem (loss 0 in 87 evaluations)

3. Task 1 – Entanglement Demonstration (Cirq + Quokka)

> Task question: Create an entangled Bell state using Cirq and run the system on the Quokka device. The Bell state is $|\psi\rangle = (1/\sqrt{2})(|00\rangle + |11\rangle)$. Generate qubits and apply the necessary gates to generate the state, and generate the necessary results. Notebook: Task_1_Bell_State/Task1_Bell_State_Cirq_Quokka.ipynb

3.1 What is a Bell state, and why is it special?

A **qubit** is a two-level quantum system whose state is $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex amplitudes with $|\alpha|^2 + |\beta|^2 = 1$ (Nielsen & Chuang, 2010; Yanofsky & Mannucci, 2008). Two qubits live in a four-dimensional space with basis $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$. The Bell state asked for in the task is

$$|\Phi^+\rangle = (1/\sqrt{2})\ (\ |00\rangle + |11\rangle\ )$$

It is **entangled**: it cannot be written as (state of qubit 0) $\otimes$ (state of qubit 1). Suppose it could; then $(a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle)$ would need $ac = bd = 1/\sqrt{2}$ but $ad = bc = 0$, which is impossible. In practice, each qubit on its own is a fair coin, but the two coins always land the same way. Einstein, Podolsky and Rosen (1935) used such correlations to question quantum theory; Bell (1964) showed they cannot be explained by hidden classical values, and loophole-free experiments have confirmed this (Hensen et al., 2015).

3.2 Which gates make the state, and why?

Two gates are enough, as taught in the course (Bahrami, 2026a). The **Hadamard (H)** gate creates an equal superposition, and the **controlled-NOT (CNOT)** gate copies the control's value into the target *for every branch of the superposition at once*, which links (entangles) the qubits.

$$H = (1/\sqrt{2})\ [\ [1,\ 1],\ [1,\ -1]\ ] \qquad \mathrm{CNOT} = [\ [1,0,0,0],\ [0,1,0,0],\ [0,0,0,1],\ [0,0,1,0]\ ]$$

Table 5. *State of the two qubits after each gate (written $|q0\ q1\rangle$)*

Step	Gate	State
0	start	$\|00\rangle$
1	H on q0	$(1/\sqrt{2})(\|0\rangle + \|1\rangle) \otimes \|0\rangle = (1/\sqrt{2})(\|00\rangle + \|10\rangle)$
2	CNOT (control q0, target q1)	$(1/\sqrt{2})(\|00\rangle + \|11\rangle) = \|\Phi^+\rangle$
3	measure both	00 with probability $\|1/\sqrt{2}\|^2 = 0.5$; 11 with probability 0.5

3.3 How did we build it in Cirq?

The Cirq code creates two `LineQubit` objects, applies `cirq.H(q0)` and `cirq.CNOT(q0, q1)`, and measures both qubits into one register. Before using the device, the notebook simulates the circuit without measurement to obtain the exact amplitudes: 0.7071 for $|00\rangle$ and $|11\rangle$ and 0 for the others, matching Table 5. The circuit is then exported with `cirq.qasm()`, cleaned by the helper and sent with 1000 shots.

```
q0, q1 = cirq.LineQubit.range(2)          # two qubits, both start in |0>
bell = cirq.Circuit(
    cirq.H(q0),                           # superposition on q0
    cirq.CNOT(q0, q1),                    # entangle q1 with q0
    cirq.measure(q0, q1, key="c"))       # read both qubits
run = qh.run_qasm(qh.clean_qasm(cirq.qasm(bell)), shots=1000)
```

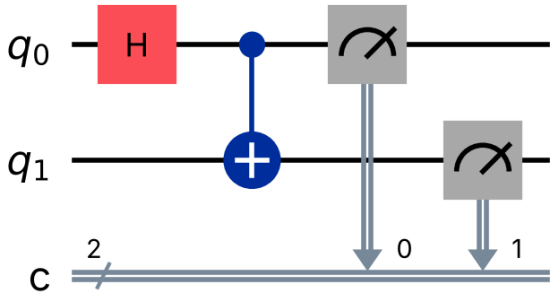


Figure 5. *Task 1 Bell-state circuit: H on q0, CNOT from q0 to q1, then measurement*

3.4 What did the measurement show?

Table 6. *Task 1 results on the Quokka device (quokka2, 4 November 2026): theory versus 1000-shot measurement*

Outcome	Theory P	Counts (of 1000)	Measured P	Difference
00	0.500	501	0.501	+0.001
01	0.000	0	0.000	0.000
10	0.000	0	0.000	0.000
11	0.500	499	0.499	−0.001

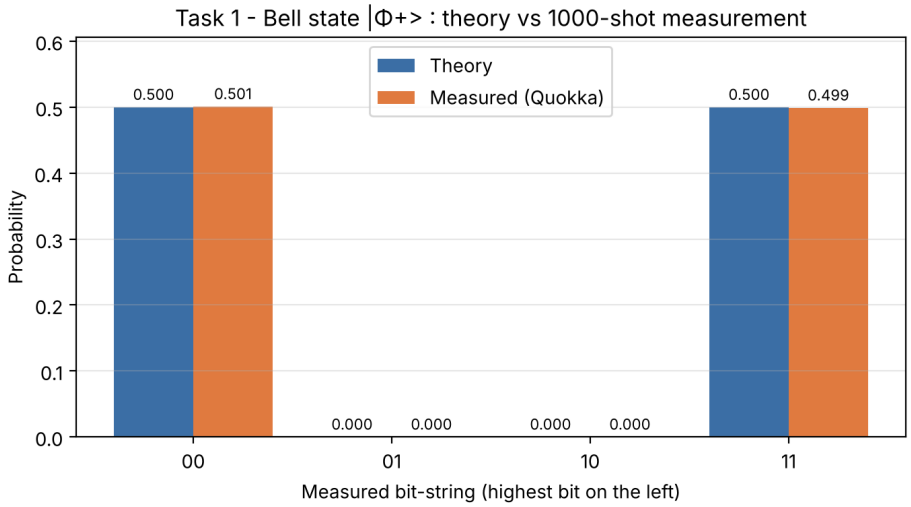


Figure 6. *Task 1 histogram: theory and measured probabilities for each outcome*

All 1000 shots were **correlated** (00 or 11), and none were 01 or 10. The 00 count of 501 is only 0.06 standard deviations above 500 (z = +0.06), and the TVD between theory and measurement is 0.001. Quokka reported 3.1 ms of processing time for the 1000 shots; the whole request took 0.63 s because of the internet round trip. Before Quokka came back online, the same QASM on the labelled local simulator gave 508 × 00 and 492 × 11 (z = +0.51), which is also within normal shot noise.

3.5 Why is the split not exactly 500/500?

Measurement is random. Each shot is an independent trial with p = 0.5, so the 00 count follows a binomial distribution with mean 500 and standard deviation 15.8. A perfect 500/500 split would itself be unusual (it happens only about 2.5% of the time). Small differences are **expected** and are not errors.

3.6 Would a real superconducting or ion-trap computer give the same result?

Not exactly. Physical devices have gate errors, decoherence and readout errors, which produce a few percent of 01 and 10 outcomes (Preskill, 2018). Quokka is designed to emulate an ideal fault-tolerant machine (UTS, 2024), and our Quokka run indeed shows zero 01/10 counts (Table 6). If 01 or 10 ever appear, the first things to check are the qubit order in the QASM and the bit order in the reply. The helper checks bit order by reading each shot as [c0, c1] and reversing it to the Qiskit convention.

3.7 What does Task 1 prove?

- The circuit H → CNOT → measure correctly prepares $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ (LO2).
- Each qubit alone looks random (50/50), but the pair is perfectly correlated, which is the measurable signature of entanglement.
- Agreement between theory and measurement on the real Quokka device is quantified (TVD = 0.001, z = +0.06), not just claimed.

4. Task 2 – Qiskit Circuits and a Custom QASM Circuit

> Task question: Using Qiskit, generate the given circuit (H on qubit 0, X on qubit 1, then block C). Measure the output on a Quokka device and print it. Then write a QASM script for a circuit of your own using different qubits and various gates, run it on Quokka and print the output. Lecturer's clarification: block C = at least 2–3 more gates of our choice, including a controlled gate such as CNOT; explain why the gates were chosen, what happens to the qubits, whether entanglement is created and what the results mean. The report should give an introduction, Circuit 1, Circuit 2, an analysis and a conclusion. Notebook: Task_2_Qiskit_QASM/Task2_Qiskit_Circuits_and_QASM.ipynb

4.1 Introduction: what are qubits and quantum circuits?

A **qubit** is the quantum version of a bit. It can be $|0\rangle$, $|1\rangle$ or a **superposition** $\alpha|0\rangle + \beta|1\rangle$, and a measurement gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. A **quantum circuit** is a list of gates applied to qubits from left to right, followed by measurements (Nielsen & Chuang, 2010). One-qubit gates rotate a single qubit (H, X, Z, S, RY). Two-qubit gates such as the **CNOT** can link qubits so that they become **entangled**, which means their joint state cannot be split into one state per qubit. Because a measurement gives a random answer, every circuit in this task was run **1000 times (shots)**, and we compare the counts with the probabilities predicted by the mathematics.

Both circuits were sent to the **Quokka device** as OpenQASM 2.0 on 4 November 2026. The device that answered was **quokka2** (Section 2.6 explains the device checks).

4.2 Circuit 1: Why did we choose these gates for block C?

The brief fixes the start of the circuit: **H on qubit 0** and **X on qubit 1**. For block C we wanted gates that tell a complete story about entanglement: **create it, change it, and measure it**. We chose four gates, two of them CNOTs:

Table 7. *Circuit 1: every gate, the qubit it acts on, and why we chose it*

Gate	Qubit	Why we chose it and what it does
H (given)	q0	Puts q0 into equal superposition $(|0\rangle + |1\rangle)/\sqrt{2}$.
X (given)	q1	Flips q1 from $|0\rangle$ to $|1\rangle$.
C1: CNOT	q0 → q1	Flips q1 only in the branch where q0 = 1. This creates entanglement: the Bell state $|\Psi^+\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$.
C2: Z	q0	Multiplies the $|1\rangle$ branch of q0 by −1. The state becomes $|\Psi^-\rangle = (|01\rangle - |10\rangle)/\sqrt{2}$. A direct measurement cannot see this change.
C3: CNOT	q0 → q1	First half of a Bell measurement: it removes the entanglement and moves the information into q0's phase.
C4: H	q0	Second half of the Bell measurement: it turns the phase (+ or −) into a 0 or 1 that can be measured.
Measure	q0, q1	Reads both qubits into c[0] and c[1].

This design is a small version of **superdense coding** (Bennett & Wiesner, 1992), where one local gate on one half of an entangled pair changes which of the four Bell states the pair is in, and a Bell measurement reads it back. To prove each claim with data, we ran **three versions**: **A-1 checkpoint** (stop after Z and measure, to show the entanglement), **A-2 full circuit**, and **A-3 control** (the full circuit without Z, to show that Z really changes the answer).

```
q0: ──H──■──Z──■──H──M
         │     │
q1: ──X──X─────X─────M
         └─ block C ─┘
```

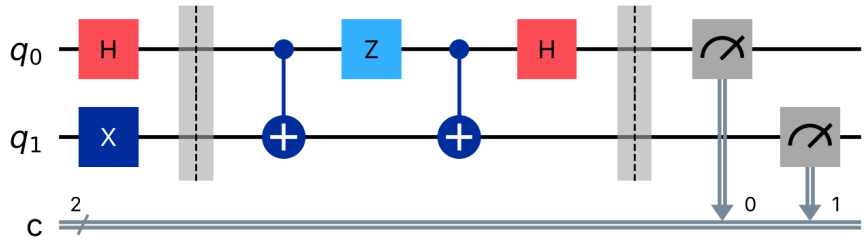


Figure 7. *Circuit 1 drawn by Qiskit: the given H and X, then block C (CNOT, Z, CNOT, H) between the barriers*

4.3 Circuit 1: What QASM code was sent to Quokka?

Qiskit built the circuit with `qc.h(0); qc.x(1); qc.cx(0, 1); qc.z(0); qc.cx(0, 1); qc.h(0)` and exported it with `qiskit.qasm2.dumps()`. The helper removed the `include` line and the barriers, so this exact text was sent:

```
OPENQASM 2.0;
qreg q[2];
creg c[2];
h q[0];          // given: superposition on qubit 0
x q[1];          // given: qubit 1 -> |1>
cx q[0],q[1];    // C1: entangle -> |Psi+>
z q[0];          // C2: phase flip -> |Psi->
cx q[0],q[1];    // C3: Bell measurement, part 1
h q[0];          // C4: Bell measurement, part 2
measure q[0] -> c[0];
measure q[1] -> c[1];
```

4.4 Circuit 1: What happens to the qubits, step by step?

Table 8. *Circuit 1: state after each step (written |q0 q1⟩) and the entanglement entropy of q0*

Step	Gate	State |q0 q1⟩	Entropy (bits)	Entangled?
0	start	$\|00\rangle$	0	no
1	H on q0, X on q1	$(1/\sqrt{2})(\|0\rangle + \|1\rangle) \otimes \|1\rangle$	0	no
2	CNOT	$(1/\sqrt{2})(\|01\rangle + \|10\rangle) = \|\Psi^+\rangle$	1	yes
3	Z on q0	$(1/\sqrt{2})(\|01\rangle - \|10\rangle) = \|\Psi^-\rangle$	1	yes
4	CNOT	$(1/\sqrt{2})(\|0\rangle - \|1\rangle) \otimes \|1\rangle$	0	no
5	H on q0	$\|1\rangle \otimes \|1\rangle = \|11\rangle$	0	no

Is entanglement created? Yes. The notebook measures entanglement with the **von Neumann entropy** of qubit 0 after qubit 1 is traced out. It is 0 for independent qubits and 1 bit for a maximally entangled pair. The entropy jumps from 0 to **1 bit** after the first CNOT, stays at 1 after Z, and falls back to 0 after the second CNOT (Table 8). So block C creates entanglement and then deliberately removes it in order to read it.

4.5 Circuit 1: What did Quokka return, and what do the results mean?

Qiskit prints bit-strings as c1c0, with qubit 1 on the left. In that order the predictions are: A-1 gives only and 10 at 50% each; A-2 gives 11 every time; A-3 gives 10 every time.

Table 9. *Circuit 1 on Quokka (quokka2), 1000 shots per run; Qiskit order c1c0*

Run	00	01	10	11
A-1 checkpoint (H, X, CNOT, Z) – theory	0	500	500	0
A-1 checkpoint – Quokka	0	532	468	0
A-2 full circuit – theory	0	0	0	1000
A-2 full circuit – Quokka	0	0	0	1000
A-3 control without Z – theory	0	0	1000	0
A-3 control without Z – Quokka	0	0	1000	0

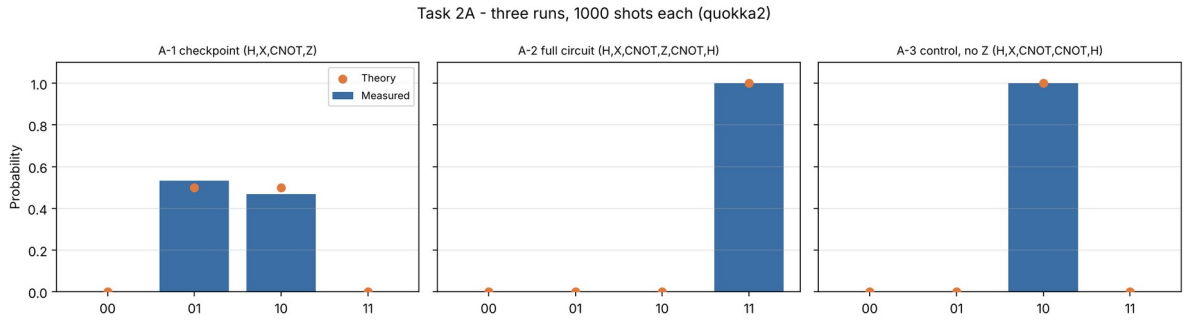


Figure 8. *Circuit 1 on Quokka: measured probabilities (bars) and theory (dots) for the three runs*

- **A-1 shows the entanglement.** Every one of the 1000 shots gave opposite bits (01 or 10), and never 00 or 11. Each qubit alone is a fair coin, but the pair always disagrees. Two independent qubits cannot do this. The split 532/468 is 2.0 standard deviations from 500 (SD = 15.8). That is the largest deviation in our project, but chance produces it in about 1 run in 20, and the TVD is still only 0.032.
- **A-2 identifies the Bell state.** All 1000 shots gave 11. A Bell measurement maps the four Bell states to four different bit-strings, and 11 is the code for $|\Psi^-\rangle$. The answer is certain, not random, because the gates after Z make the amplitudes **interfere**.
- **A-3 proves the phase is real.** Without Z, all 1000 shots gave 10. The only difference from A-2 is one Z gate, and Z never changes a probability directly (A-1 looks the same with or without it). It changed the **phase**, and interference turned that phase into a different measured bit.

4.6 Circuit 2: Why did we design a quantum half adder?

The lecturer asked for a **completely different** circuit with **different gates and a different qubit configuration**. Circuit 2 uses **four qubits** (not two), and five gate types: H, Z, S, Toffoli (CCX) and CNOT. We chose a circuit with a clear purpose that we could check fully: a **half adder**, the basic building block of computer arithmetic, which returns SUM = A XOR B and CARRY = A AND B. Quantum adders built from CNOT and Toffoli gates are well known (Vedral et al., 1996). Our twist is to place the inputs in **superposition**, so one circuit holds all four additions (0+0, 0+1, 1+0, 1+1) at the same time. Each shot then reveals one complete, correct row of the truth table.

Table 10. *Circuit 2: qubits and gates (file custom_half_adder.qasm)*

Qubit	Role	Gates	Why this gate
q0	input A	H, Z	H makes A = 0 and 1 at once; Z adds a −1 phase, to show that phase is invisible to a direct measurement
q1	input B	H, S	H for superposition; S adds an i phase (the same lesson)
q2	CARRY	CCX (Toffoli)	Flips to 1 only if A = 1 AND B = 1 (a three-qubit controlled gate)
q3	SUM	CX, CX	Two CNOTs give SUM = A ⊕ B

4.7 Circuit 2: What is the QASM code?

```
OPENQASM 2.0;
include "qelib1.inc";
qreg q[4];  creg c[4];
h q[0];  h q[1];          // A and B in superposition
z q[0];  s q[1];          // phase gates: change amplitudes, not probabilities
ccx q[0],q[1],q[2];       // CARRY = A AND B
cx q[0],q[3];  cx q[1],q[3];   // SUM = A XOR B
measure q[0] -> c[0]; measure q[1] -> c[1];
measure q[2] -> c[2]; measure q[3] -> c[3];
```

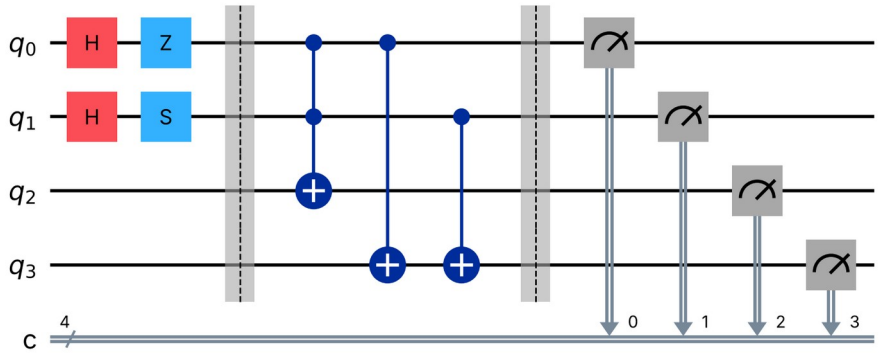


Figure 9. *Circuit 2 (quantum half adder) loaded from our QASM file and drawn by Qiskit*

4.8 Circuit 2: What does the theory predict?

After the two H gates the inputs are $(1/2)(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$. The Z and S gates multiply some amplitudes by −1 or i. The state vector in the notebook shows amplitudes 0.5, −0.5i, −0.5 and 0.5i, but every probability is still $|\pm 0.5|^2 = |\pm 0.5i|^2 = 0.25$. The Toffoli and CNOT gates then write CARRY and SUM. Writing strings as c3c2c1c0 = SUM, CARRY, B, A, only four strings are possible: **0000, 1001, 1010 and 0111**, each with probability 0.25.

4.9 Circuit 2: Can Quokka run every gate in our design?

Not directly, and checking this was an important engineering step. The official *Quokka docs* notebook lists the QASM gates the Quokka simulator understands: **h, x, y, z, rx, ry, rz, cx, swap, cphase and U** (Quokka Computing, n.d.-b). Our design uses **s** and **ccx** (Toffoli), which are not on the list. Real quantum devices are the same: they run only a small native gate set, and a Toffoli must be decomposed into one- and two-qubit gates (Barenco et al., 1995). The shared helper therefore

rewrites every circuit into Quokka's gate set before it is sent: S becomes RZ(π/2), and the Toffoli becomes CNOT, H and RZ(±π/4) gates. The notebook proves that the rewritten circuit has exactly the same probabilities. The cost is visible: our 7-gate design became **21 gates** (8 RZ, 8 CX, 4 H and 1 Z), and on noisy hardware every extra gate adds error.

4.10 Circuit 2: What did Quokka return?

Table 11. *Circuit 2 on Quokka (quokka2), 1000 shots: every observed string is a correct addition*

String	A	B	SUM	CARRY	Theory P	Counts	Measured P
0000	0	0	0	0	0.25	261	0.261
1001	1	0	1	0	0.25	250	0.250
1010	0	1	1	0	0.25	247	0.247
0111	1	1	0	1	0.25	242	0.242

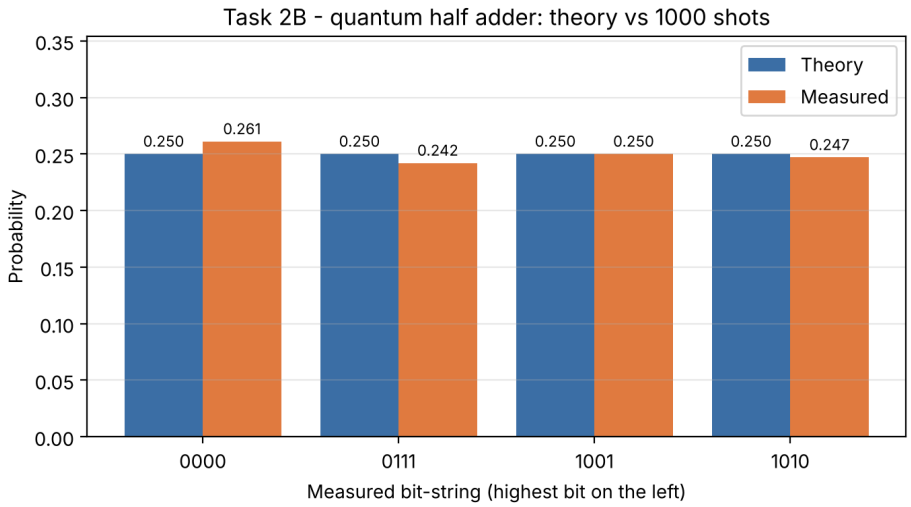


Figure 10. *Circuit 2 on Quokka: the four rows of the half-adder truth table*

All **1000 of 1000** shots were correct additions. The four rows appeared 242–261 times (expected 250 ± 13.7), and the TVD is 0.011. The output means: in each shot the device picked one input pair at random, and the SUM and CARRY qubits always held the correct answer for that pair.

4.11 Analysis: How do the two circuits compare?

Table 12. *Circuit 1 and Circuit 2 side by side*

Feature	Circuit 1 (given start + block C)	Circuit 2 (our QASM half adder)
Qubits	2	4
Gate types	H, X, CNOT, Z	H, Z, S, CNOT, Toffoli
Superposition	q0 after H	A and B after H (4 inputs at once)
Entanglement	Yes, maximal (1 bit) after the first CNOT, then removed by the Bell measurement	Yes: before measurement the output qubits are correlated with the inputs (e.g. SUM = A ⊕ B)
Role of phase	Decisive: Z changes the result from 10 to 11 through interference	Invisible: Z and S change amplitudes, but no gate after them makes them interfere

Feature	Circuit 1 (given start + block C)	Circuit 2 (our QASM half adder)
Measured result	Deterministic (A-2, A-3) or 50/50 opposite bits (A-1)	Four outcomes, about 25% each
What it demonstrates	creating, changing and reading a Bell state	parallel evaluation of a logic function, and the cost of gate decomposition

4.12 Analysis: What do superposition and entanglement look like in the data?

- **Superposition gives randomness, not parallel answers.** Circuit 2 holds four additions at once, but each shot shows only one of them. Reading all four rows took many shots, while a classical half adder needs only four runs. A quantum advantage needs interference that concentrates probability on useful answers, as in Grover's or Shor's algorithms.
- **Entanglement shows up as correlation.** In A-1, each bit alone is random but the pair always disagrees. In Circuit 2, SUM and CARRY always agree with A and B. These correlations are present in every one of the 1000 shots.
- **Interference turns phase into information.** The same Z gate is invisible in A-1 and in Circuit 2, but decisive in A-2. The difference is whether a later gate (here the CNOT and H of the Bell measurement) makes the amplitudes interfere.
- **Probabilities are estimates.** Measured probabilities differ from theory by a few percent (up to 2 SD in A-1), exactly as binomial shot noise predicts. Outcomes with theoretical probability 0 never appeared in any Task 2 run, which is the strongest evidence that the circuits are correct.

4.13 Conclusion: What did Task 2 teach us?

Task 2 taught us how to build a circuit with a purpose and then prove the purpose with measurements. Circuit 1 showed that one CNOT creates a maximally entangled pair, that one Z gate changes the pair into a different Bell state without changing any single measurement, and that a Bell measurement reads that hidden difference with certainty (1000/1000). Circuit 2 showed superposition used for parallel evaluation, three-qubit control logic, and the practical need to translate gates into a device's native set. Both circuits were written in OpenQASM, the common language that let the same text run in Qiskit, on our computer and on Quokka (Cross et al., 2017).

5. Task 3 – Investigating a Quantum Code: Quantum Tic-Tac-Toe

> Task question: Complete the provided code by filling in missing instructions or comments; play multiple games and analyse the generated code; as a quantum software engineer, explain the purpose, functionality, gameplay dynamics and how circuits are generated (Word document); and describe each quantum gate used in a separate section. Notebook: Task_3_Quantum_TicTacToe/Task3_Quantum_TicTacToe_Completed.ipynb (original kept unchanged beside it)

5.1 What is the purpose of the program?

The program, supplied as Lab 5 (Bahrami, 2026b), is a **teaching game**. Quantum games have long been used to make superposition and measurement intuitive (Goff, 2006). Here every tile of the 3 × 3 board is **one qubit**, and every move adds a **gate** to a nine-qubit circuit. Nobody owns a tile until the end, when the circuit is measured once: a 0 makes the tile X's and a 1 makes it O's. Playing therefore means *designing a quantum circuit*, and the result is probabilistic. The game makes three ideas visible: superposition (undecided tiles), unitary gates as moves, and measurement collapse.

5.2 How is the code organised?

The code has two classes. **`Board`** is the quantum part: it owns the `QuantumCircuit(9, 9)`, adds gates in `make_move`, measures in `measure`, and counts winning lines in `countWinners`. **`Game`** is the classical user interface: it creates buttons with `ipywidgets`, reacts to clicks in `handle_game`, and redraws the board. This separation of quantum logic from the user interface is good software design. It is also how real quantum applications are built: a classical program prepares and sends circuits and then interprets the results.

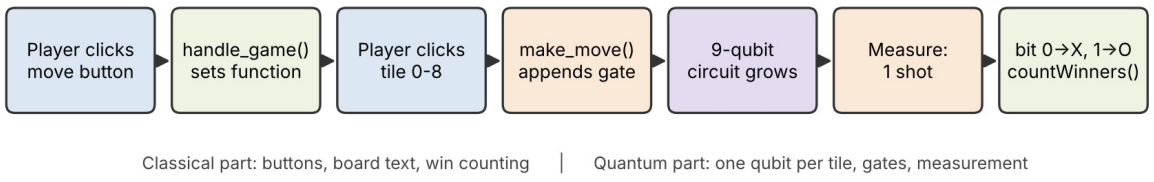


Figure 11. *How one move travels through the code: from a click to a gate, and from measurement to the score*

5.3 What was missing, and what did we complete or fix?

Table 13. *Code completion and fixes (marked [ADDED], [FIXED] or [CHANGED] in the notebook)*

#	Location	Problem in supplied code	Our completion
1	make_move – Not	comment only, no gate	self.qc.x(cell)
2	make_move – O	comment only, no gate	self.qc.ry(+π/4, cell) (toward |1⟩ = O)
3	make_move – X	comment only, no gate	self.qc.ry(−π/4, cell) (toward |0⟩ = X)
4	make_move – SWAP	comment only, no gate	self.qc.swap(cell, self.target)
5	countWinners	winners = () – nobody could ever win	the 8 lines: 3 rows, 3 columns, 2 diagonals
6	SWAP branch	dead inner if; crash if no first tile	guard target ≥ 0; reset target after swap
7	new()	empty tile '' but __init__ uses ' '	made consistent
8	imports / grid	google.colab widgets only work in Colab	ipywidgets HBox/GridBox (VS Code, Jupyter,

#	Location	Problem in supplied code	Our completion
			Colab)
9	screen updates after a click	callback output only shown in Colab; in Jupyter clicks seemed to do nothing	draw every screen inside an ipywidgets Output widget

An important finding: the supplied comments had the rotation directions swapped. `measure()` maps bit 0 to X and bit 1 to O, but the comments said the O button rotates toward $|0\rangle$ and the X button toward $|1\rangle$. Followed literally, pressing **O** would help player **X**. The game's own rules say "O turns the tile toward a tile owned by O", so we implemented O as a rotation toward $|1\rangle$ and X toward $|0\rangle$ and corrected the comments. Finding a mismatch between comments, code and specification, and resolving it against the specification, is a normal part of investigating someone else's code.

5.4 How does a move become a gate? (Circuit generation)

1. **Start of a round.** `__init__` (and `new()`) adds `reset` and **H** to all nine qubits, then a barrier. Every tile is $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, i.e. 50% X and 50% O.
2. **Choose a move.** Clicking Not, O, X or SWAP raises a `ValueError` when `int(b.description)` is tried, and the `except` branch stores the move in `board.function`. This is a slightly unusual way to tell text buttons from number buttons.
3. **Choose a tile.** Clicking 0–8 calls `make_move(cell)`, which appends one gate to `self.qc`. For SWAP, the first tile click sets `board.target` and the second click applies the swap.
4. **The circuit grows.** After each move the circuit is redrawn. The text on the board is only a move history; as the original notes say, "the game circuit is the true state of the game".
5. **Measure.** Barrier and nine `measure` instructions are added, and the circuit runs **once** (1 shot) on AerSimulator. Bit i of the result decides tile i, read from the right-hand end of Qiskit's string (`output[9-1-i]`).
6. **Score.** `countWinners` checks the eight lines for each player. Both players can win lines in the same round, because the tiles are decided independently.

5.5 How does a rotation move change the odds?

On the Bloch sphere, $|+\rangle$ sits at angle $\theta = \pi/2$ from $|0\rangle$, and $P(X) = P(0) = \cos^2(\theta/2)$. Each X move applies $R_y(-\pi/4)$, which reduces θ by $\pi/4$ (Figure 12). One X move gives $P(X) = \cos^2(\pi/8) = 0.854$, two moves give 1.000, and **a third move over-rotates** back to 0.854. Moves are rotations, not counters, so "too many" moves undo themselves. This is a quantum effect that classical tic-tac-toe does not have.

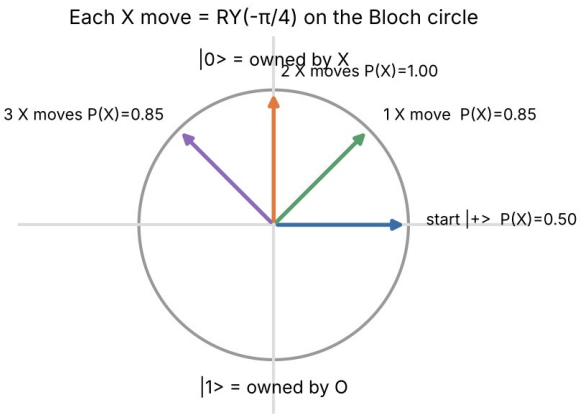


Figure 12. *Effect of repeated X moves on one tile (x–z plane of the Bloch sphere)*

5.6 What happened when we played? (Four recorded games)

Button clicks cannot be saved in a notebook, so we replayed four games as scripts that call exactly the same `make_move` and `measure` code with fixed seeds. For each game we saved the circuit picture, the one-shot result, and a **1000-shot study** of the same circuit, which estimates each tile's chance of becoming X and each player's chance of winning (more X lines than O lines).

Table 14. *Recorded games: moves, generated gates, the one-shot result and 1000-shot win chances*

Game	Moves	Gates added	1-shot lines X:O	X wins /1000	O wins /1000	Draws / 1000
1 Baseline	none	9 H only	4 : 0	392	405	203
2 Diagonals	X on 0, 4, 8; O on 2, 6	+5 RY	4 : 0	705	210	85
3 NOT test	X, X on 4; Not on 4; O on 0; Not on 1	+3 RY, 2 X	0 : 2	135	694	171
4 SWAP	X, X on 0; O, O on 4; SWAP 0 ↔ 8; X × 3 on 2	+7 RY, 1 SWAP	0 : 2	404	289	307

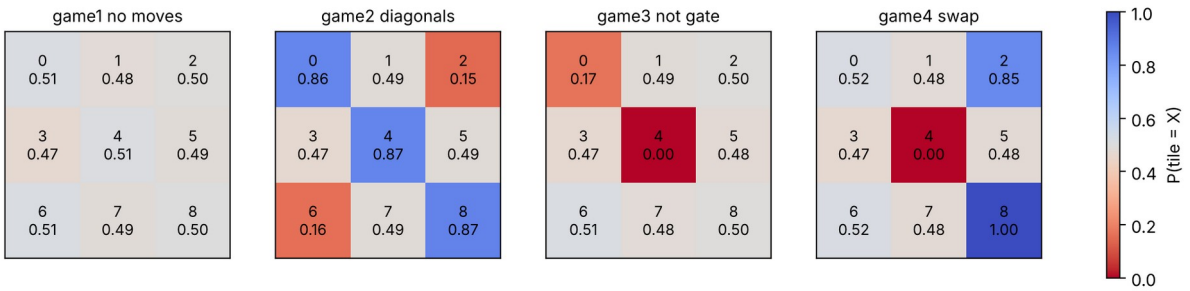


Figure 13. *Chance that each tile becomes X after each recorded game (1000 shots; blue = X likely, red = O likely)*

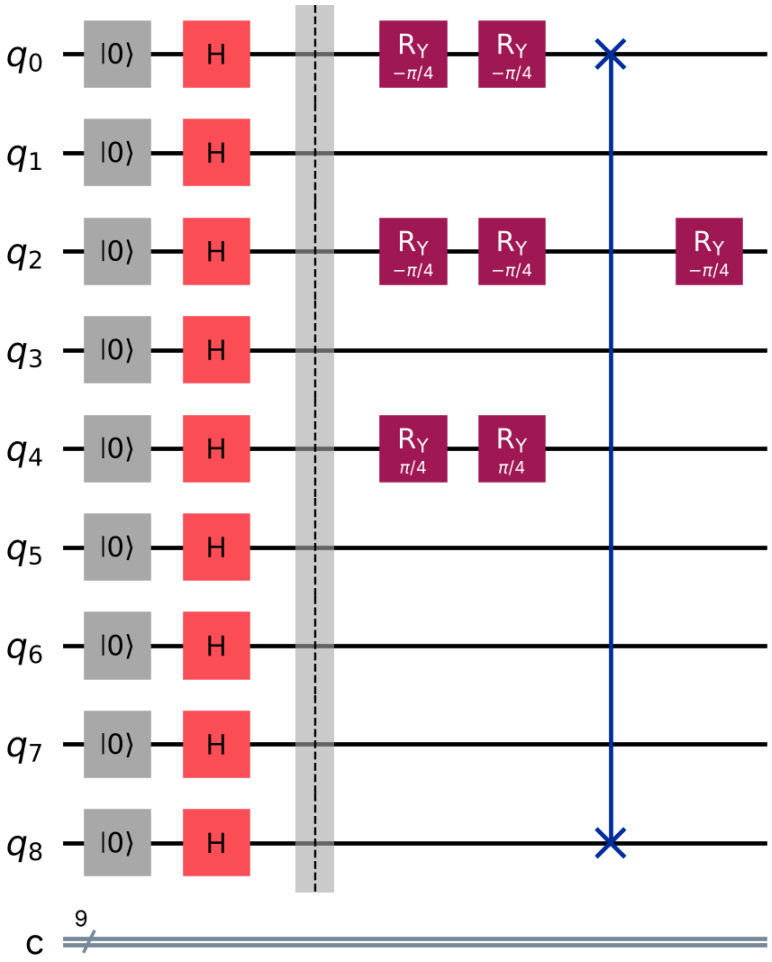


Figure 14. *Circuit generated by Game 4: H start layer, RY moves, and the SWAP between tiles 0 and 8*

5.7 What do the games show? (Analysis)

- **Game 1 (baseline):** with no moves every tile is 50/50, so X and O win about equally often (392 vs 405). The one-shot result (X 4 lines) was simply luck. A single game says little; the 1000-shot study shows the real odds.
- **Game 2:** three X moves on the main diagonal (P(X) ≈ 0.86 each) and two O moves on the other corners raised X's win chance from 39% to 71%. Strategy works, but only in probability.
- **Game 3 (NOT gate):** two X moves made tile 4 certainly X, and NOT then made it certainly O (P(X) = 0.00). NOT on the *undecided* tile 1 did **nothing** (0.49), because $|+\rangle$ is an eigenstate of X: $X|+\rangle = |+\rangle$. This matches the rule "if the tile is not owned, Not does nothing", and it is explained by linear algebra, not by an if-statement.
- **Game 4 (SWAP):** tile 0 was made certainly X and then swapped with tile 8. Afterwards tile 8 is certainly X (1.00) and tile 0 is back to 50/50, because SWAP moved the quantum *states*, not labels. Tile 2 received three X moves and shows the over-rotation (0.85, not 1.00).
- **Circuit growth:** every move adds one gate and does not remove any, so circuit depth rose from 2 to 5. On noisy hardware, longer games would therefore be less reliable.

5.8 Did we also play the game by clicking the buttons?

Yes. To check that the completed game really works for a player, we opened the notebook in **Jupyter Notebook 7** (the environment used in the lectures), ran it, and played three games by clicking the move and tile buttons. Each **Measure** click runs the game circuit once, so every result is a genuine single random outcome.

Table 15. *Three games played interactively in Jupyter Notebook (31 October 2026)*

Game	Buttons clicked	Final board (rows)	Result and what it shows
A	X 4, X 4, O 0, O 8, Not 2	X O O / O X X / O X O	Draw 0–0. Tile 4 (two X moves) was certain X. Tile 0 had one O move (85% O) yet became X: a 15% outcome happened.
B	X 0, X 4, X 8, O 2, O 6	X O O / X X O / O O X	X wins the main diagonal 0–4–8, the three tiles nudged toward X.
C	O 4, O 4, X 1, Not 4, SWAP 0 ↔ 8	X X X / X X O / O O X	X wins 2 lines. Two O moves made tile 4 certain O, then NOT made it certain X, exactly as the theory predicts.

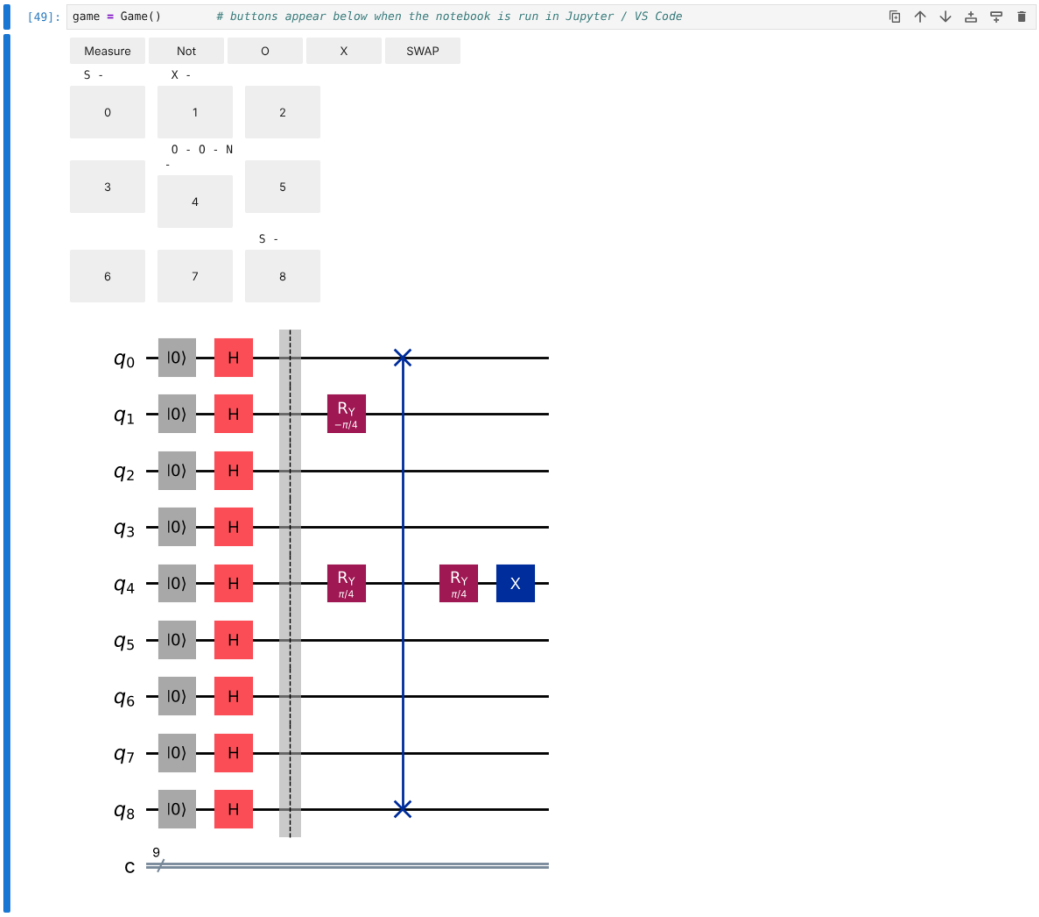


Figure 15. *Game C in Jupyter before Measure: the board shows the move history, and the circuit shows RY, X and the SWAP between tiles 0 and 8*

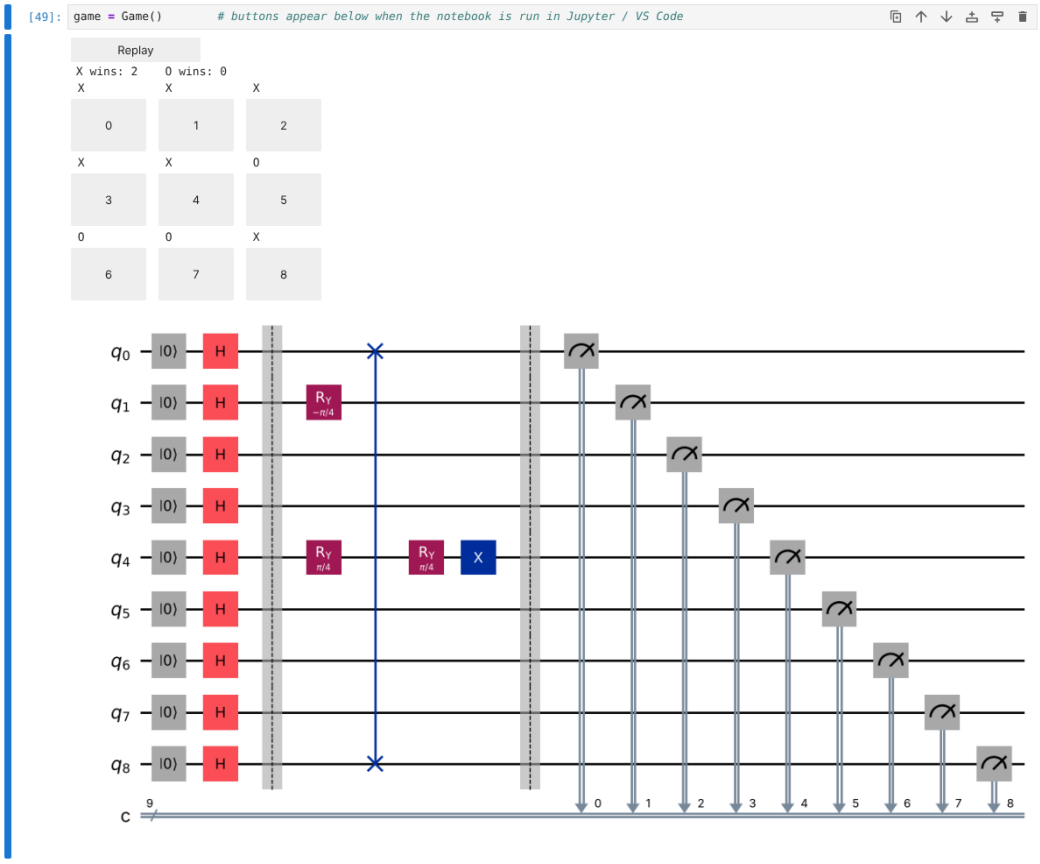


Figure 16. *Game C after Measure: one shot decided every tile; X wins 2 lines*

Playing also exposed a **portability bug**. In the supplied code, every screen update after a click (board, circuit, score) is printed from inside the button's callback. Google Colab shows that output, but Jupyter Notebook and VS Code discard it, so clicks *appeared to do nothing*. We fixed this by drawing all game screens inside an `ipywidgets.Output` widget (marked `[CHANGED]`), after which the game worked in all three environments.

5.9 Can the game circuit run on Quokka?

Yes. The notebook removes the `reset` instructions (qubits already start at $|0\rangle$), translates the Game 2 circuit to H, RY, X and CX gates, exports it to QASM and sends it with 1000 shots. On the Quokka device (quokka2, 4 November 2026) the per-tile chances of X were 0.86, 0.49, 0.14, 0.48, 0.85, 0.50, 0.15, 0.49 and 0.86, the same pattern as Figure 13 and within shot noise of the theory (0.854 for one X move, 0.146 for one O move, 0.5 for untouched tiles). Quokka needed 3.3 ms for the 1000 shots of this nine-qubit circuit. This links the game back to the device used in Tasks 1 and 2.

5.10 Which quantum gates does the game use? (Gate section)

Table 16. *Quantum gates used in circuit generation: characteristics and function in the game*

Gate	Matrix / action	Key characteristic	Function in this game
H (Hadamard)	$(1/\sqrt{2})[[1,1],[1,-1]]$; $\|0\rangle \to \|+\rangle$	creates equal superposition; H·H = I	start layer: every tile undecided (50/50)
$R_y(\theta)$	$[[\cos\theta/2, -\sin\theta/2],[\sin\theta/2, \cos\theta/2]]$	rotation about the y-axis; real amplitudes; changes probabilities smoothly	X move: $R_y(-\pi/4)$ toward $\|0\rangle$; O move: $R_y(+\pi/4)$ toward $\|1\rangle$
X (NOT)	$[[0,1],[1,0]]$; $\|0\rangle \leftrightarrow \|1\rangle$	bit flip; $X\|+\rangle = \|+\rangle$	Not move: flips ownership of a

Gate	Matrix / action	Key characteristic	Function in this game
		(eigenstate)	decided tile
SWAP	exchanges two qubits' states (= 3 CNOTs)	two-qubit, permutes $\|01\rangle \leftrightarrow \|10\rangle$	SWAP move: exchanges two tiles
reset	sets qubit to $\|0\rangle$ (not unitary)	non-reversible initialisation	clean start of each round
measure	projects onto $\|0\rangle$ or $\|1\rangle$	irreversible; Born rule	decides each tile's owner (0 → X, 1 → O)
barrier	no operation	visual / compiler boundary	separates start, moves and measurement in the drawing

5.11 How do these gates fit the wider theory of quantum logic gates?

Every quantum gate is a **unitary matrix** ($U^\dagger U = I$), so every gate is **reversible**: applying $U^\dagger$ undoes it. `reset` and `measure` in the table above are the exceptions; they are not gates in this strict sense because they cannot be undone, and measurement follows the Born rule (Nielsen & Chuang, 2010; "Quantum logic gate," n.d.). Gates in sequence combine by **matrix multiplication**, written in the reverse of the drawing order (X then Y gives the matrix YX), and gates on different qubits combine by the **tensor product**. That is how the nine-qubit game state is built from nine single-qubit operations.

A gate set is **universal** if it can approximate any unitary operation; examples are {H, T, CNOT}, the Clifford+T set, or arbitrary rotations plus CNOT (Barenco et al., 1995; "Quantum logic gate," n.d.). This idea explains two of our findings:

- **The game's gates are not universal and cannot entangle tiles.** H, R_y and X keep every amplitude real, and SWAP only moves states between tiles. The game has no entangling gate such as CNOT, so its nine tiles always stay independent. Adding a CNOT move ("link two tiles") would make the gate set universal and allow entangled tiles.
- **Quokka's gate set is universal.** Its QASM gates include Rx, Ry, Rz and CNOT, which form a universal set. That is why `to_quokka_gates()` can always rewrite S, T or the Toffoli into supported gates (Task 2B). The rewritten circuit can differ only by a **global phase**, which never changes a measurement result.

5.12 What are the strengths and limits of the program?

Strengths: the program is short, has clear quantum/classical separation and a good visual circuit, and it demonstrates superposition, rotation and measurement in a playful way. **Limitations:** (1) tiles are never *entangled*, because the only two-qubit gate is SWAP, which keeps tiles independent; adding a CNOT move would create linked tiles and is a natural improvement. (2) One-shot measurement makes single games very random. (3) The board text is "not 100% accurate", so it can mislead players. (4) Using exceptions to tell buttons apart is fragile. (5) There is no turn order, so one player can make every move. Despite these limits, as a teaching tool it shows clearly that a quantum program is a circuit that is built step by step and only gives answers when measured.

6. Task 4 – Machine Learning Quantum Analysis

> Task question: Analyse the code (circuit and problem formulation; data input; quantum functionality, excluding the optimiser); execute it, monitor the output at each iteration and plot iteration vs output and iteration vs time; modify it to run locally without the quantum computer or circuit and document differences in efficiency and effectiveness; submit the modified .ipynb and a report. Notebook: Task_4_Quantum_ML/Task4_Quantum_ML_Modified.ipynb (original kept beside it)

6.1 What problem does the code solve? (Problem formulation)

The supplied Lab 10 notebook (Bahrami, 2026c) trains a quantum circuit to classify tiny 2 × 2 black-and-white images into two classes, which the code calls **stripes** (label 0) and **bars** (label 1). `generate_data(2)` creates all line patterns and removes the all-black and all-white images, because they belong to both classes. Four images remain: [1,0,1,0] and [0,1,0,1] (label 0), and [1,1,0,0] and [0,0,1,1] (label 1). Three are used for training and one for testing (75/25 split).

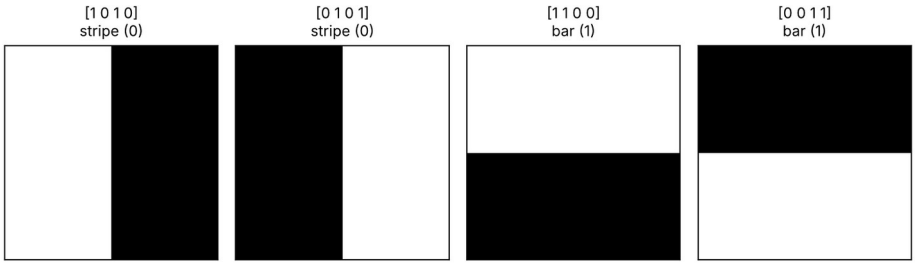


Figure 17. *The four images, their pixel vectors and labels*

The model output is the probability that the last qubit is measured as 1, estimated from shots: $\hat{y}(\theta, x) \approx$ (number of 1s) / shots. The **loss** minimised is the mean absolute error over the training images:

$$L(\theta) = (1/N)\ \Sigma_n \mid \hat{y}(\theta, x_n) - y_n \mid$$

A good model gives $\hat{y} \to 1$ for bars and $\hat{y} \to 0$ for stripes; an image is classified as a bar when $\hat{y} > 0.5$.

6.2 Which part of the code is the input to the circuit?

The input is the loop at the top of `create_circuit`: for every pixel i, `if data[i] > 0.5`, an **X gate** is added to qubit i. Each pixel is one qubit, and a white pixel flips its qubit from $|0\rangle$ to $|1\rangle$. This is **basis encoding**: the image becomes the starting basis state, e.g. [1,0,1,0] → $|q3q2q1q0\rangle = |0101\rangle$ (Schuld & Petruccione, 2021). The data enters only here; everything after it is the trainable model.

6.3 How does the circuit work? (Quantum part, excluding the optimiser)

The trainable part is built from **blocks**. `block(angles, qubits)` applies $R_Y(\theta_a)$ to one qubit, $R_Y(\theta_b)$ to another, and then a CNOT between them. `generate_pairs` splits the qubits in half again and again and joins the halves, giving the pairs (0,1), (2,3) and then (1,3). Information from all four pixels therefore flows into **qubit 3** in only $\log_2 4 = 2$ layers. This is a **tree tensor network (TTN)** classifier of the kind studied by Grant et al. (2018) and Huggins et al. (2019). With 3 blocks × 2 angles there are **6 trainable parameters**, and only qubit 3 is measured.

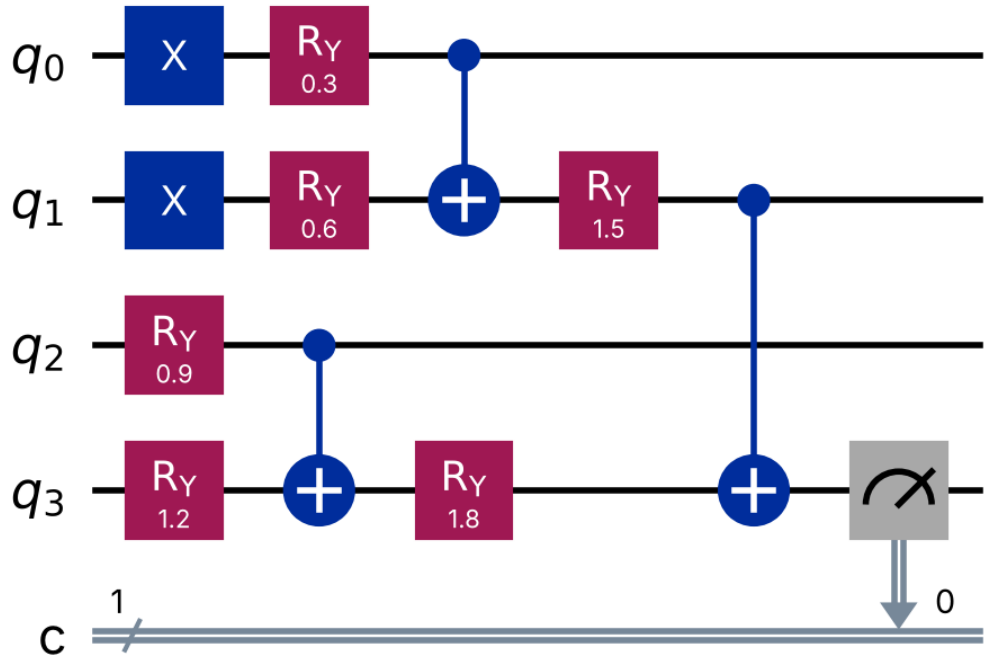


Figure 18. *Classifier circuit for image [1,1,0,0]: X gates encode the pixels, then three RY–RY–CNOT blocks form a tree into q3*

The RY gates give each block adjustable 'weights', while the CNOTs make the output depend on *combinations* of pixels. If a control qubit is in $|1\rangle$, the CNOT flips the target, so the final value of q3 behaves like an XOR (parity) of pixels that is smoothly adjusted by the angles. This structure turns out to be the key to the results below.

6.4 What did we add to monitor every iteration?

We modified the SPSA optimiser (Spall, 1992) without changing its algorithm. SPSA estimates the gradient from just **two** loss evaluations per iteration, whatever the number of parameters, which suits noisy, expensive quantum evaluations. We added a timer around each update and, after each update, one extra evaluation of the training loss, the **output measure**. This monitoring evaluation is timed separately so it does not inflate the iteration time. Settings were kept as in the original ($a = 1$, $c = 0.5$, $\alpha = 0.602$, $\gamma = 0.101$, 100 iterations), except that the shots per circuit were raised from 100 to the group's agreed **1000**, and a fixed seed (2026) makes runs repeatable. Each iteration runs 3 evaluations × 3 training images = 9 circuits, so the run executed **900 circuits** in total.

6.5 What do the two required graphs show?

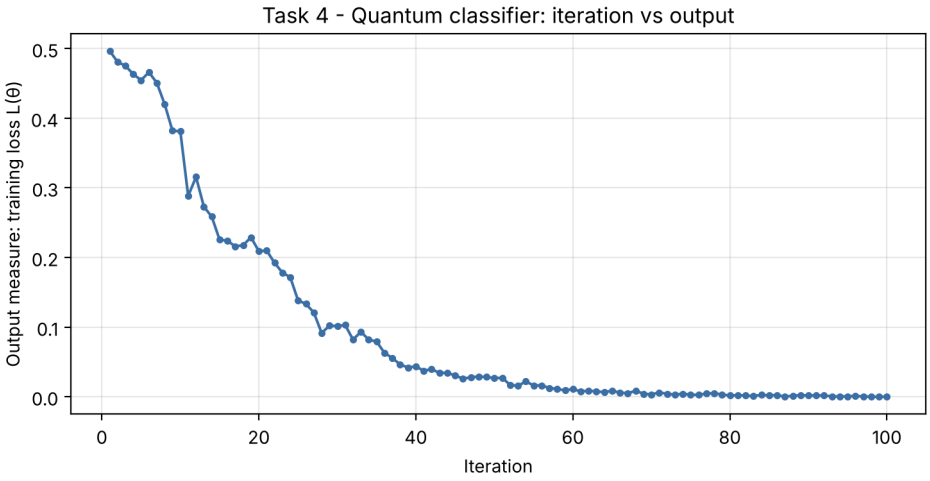


Figure 19. *Required graph 1: iteration number versus output measure (training loss)*

Table 17. *Excerpt from the 100-iteration log (task4_quantum_iterations.csv)*

Iteration	Output (training loss)	Note
1	0.497	random start: no better than guessing
10	0.381	
20	0.209	
30	0.102	
38	0.046	first iteration below 0.05
50	0.027	
70	0.003	
100	0.000	end of training

Graph 1 (run on the Quokka device (quokka2)) shows the loss falling from about 0.50, which is equal to random guessing, to 0.000 by iteration 100. It fell quickly in the first 40 iterations and then flattened. The curve is not perfectly smooth. Each loss value is itself an estimate from 1000 shots per image, and SPSA's random ±1 directions sometimes give a slightly worse step (e.g. iterations 10–11 and 15–20). The step size $a_k = 1/(k+1)^{0.602}$ shrinks over time, which explains the slow final convergence.

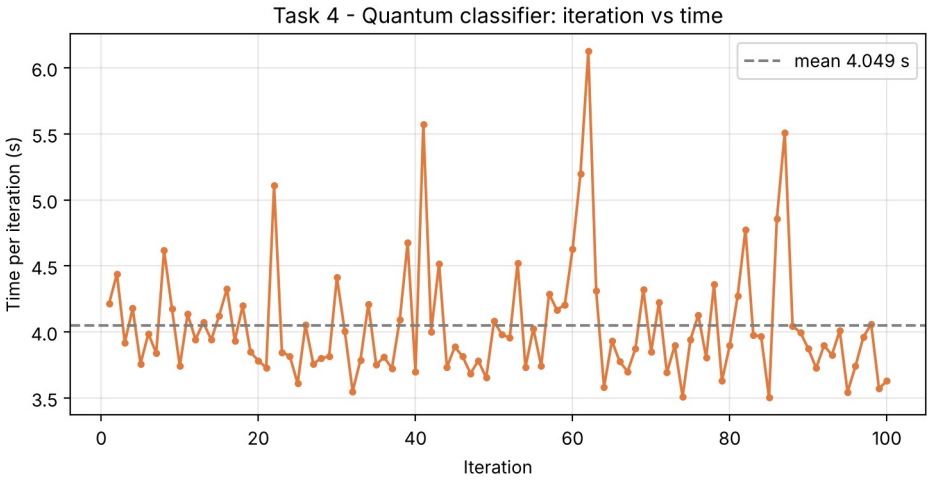


Figure 20. *Required graph 2: iteration number versus time per iteration*

Graph 2 shows an almost **constant** time per iteration of **4.05 s** on average (range 3.50–6.13 s). This is expected: every iteration runs the same number of circuits (6 for the update) with the same size and shot count, so the cost does not depend on how good the model is. Because the circuits ran on Quokka, almost all of this time is the internet round trip for each of the six requests; Quokka itself needs only a few milliseconds per circuit (Section 7). The spikes are slower network responses.

6.6 How did we make the code run without the quantum computer or circuit?

The brief can be read in two ways, so we built **both** and added one fairer baseline. Every version uses the same loss, the same SPSA settings and the same random seed, so only the model changes:

- **Local-A: no quantum computer.** The same circuit is simulated exactly with our own NumPy state-vector code: 16 complex amplitudes, with gates applied as matrices. It computes P(q3 = 1) exactly, so there is no shot noise. It was verified against Qiskit (both gave 0.500625 for a random test case).
- **Local-B: no quantum circuit.** A classical logistic regression $\hat{y} = \sigma(w \cdot x + b)$ with 5 parameters, close to the circuit's 6.
- **Local-C: a fair classical baseline.** Logistic regression with pairwise pixel products added (4 + 6 features, 11 parameters). This is the standard classical way to model interactions.

6.7 What were the differences in efficiency and effectiveness?

Table 18. *Quantum circuit versus local versions (100 SPSA iterations, seed 2026). Accuracy uses each trained model's exact output; times vary slightly between runs*

Version	Total time (s)	Time / iteration (s)	Circuit runs	Final loss	Test acc.	All 4 images
Quantum circuit (1000 shots, Quokka quokka2)	606.2	4.049	900	0.000	1/1	4/4
Local-A: same circuit, exact NumPy	0.18	0.0012	0	0.0003	1/1	4/4
Local-B: classical linear	0.007	0.00005	0	0.296	0/1	3/4
Local-C: classical + pairwise features	0.026	0.00019	0	0.206	0/1	3/4

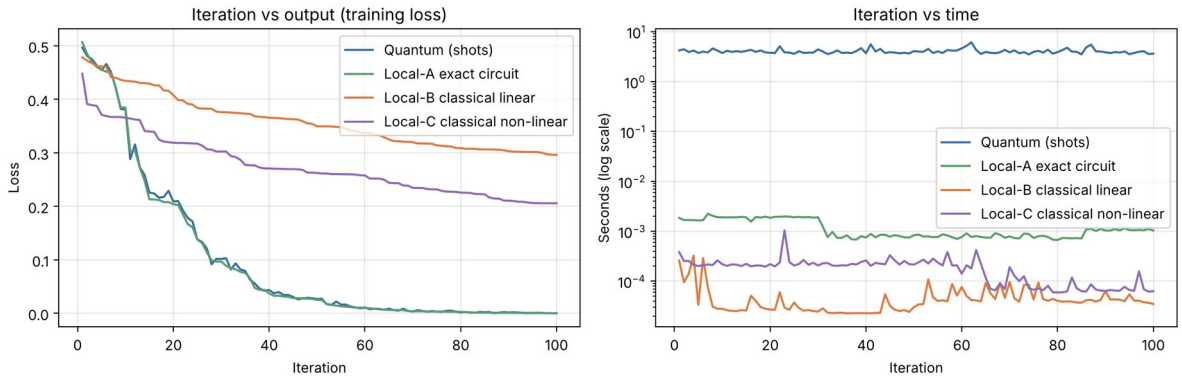


Figure 21. *All versions: loss per iteration (left) and time per iteration on a log scale (right)*

Efficiency. Removing the quantum device made each iteration about **3490 times faster** (4.049 s → 0.0012 s). A shot-based run needs 1000 samples per circuit, QASM parsing and, on Quokka, one network request per circuit (900 in total), whereas the exact simulation needs one small matrix calculation. Local-A and the quantum run followed almost the same loss curve, so the shot noise slowed the computer, not the learning. This advantage disappears as problems grow: exact simulation needs 2^n amplitudes, so 50 qubits would need about 10^{15} complex numbers, which is why real quantum hardware becomes useful only at scale (Section 7.4 measures where this begins for Quokka).

Effectiveness. The circuit (both quantum and Local-A) reached zero training loss and also classified the **unseen test image** correctly. Both classical models fitted the training images but **failed the test image**, and their losses stayed high (0.30 and 0.21). The reason is the shape of the data. In each class the two images are exact opposites, so the class sums are identical and **no straight line can separate the classes**. Like XOR, this is the classic limit of linear models (Minsky & Papert, 1969). In fact, the label equals $\mathbf{x_1}$ **XOR** $\mathbf{x_3}$ (Figure 22).

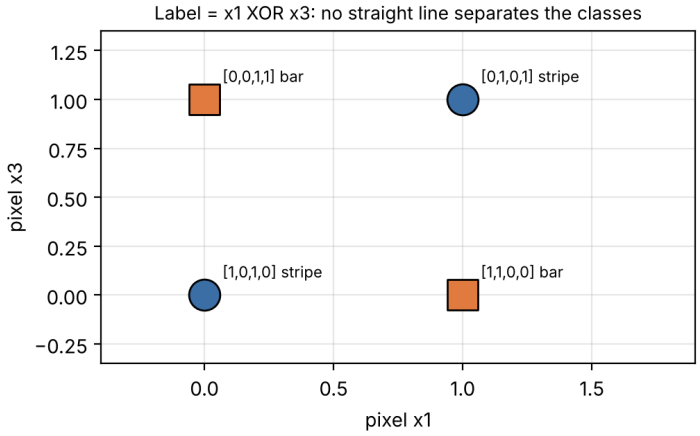


Figure 22. *The data is XOR-shaped: label = x1 XOR x3, so a linear classifier cannot separate it*

Local-C has the right *kind* of features, but the test image's key feature (the product $x_0 \cdot x_1$) is zero in all three training images, so its weight is never learned. The circuit generalised because its CNOT tree computes parity naturally. Its built-in bias (an XOR-like structure) happens to match this problem. This is an honest, limited claim: with one test image the test score is weak evidence, and a classical model designed with XOR features would also succeed. The fair conclusion is that the circuit's **structure**, not quantum speed, made it effective here.

6.8 Did anything not work as expected?

Yes. The original warm-up (one qubit learning to output 1 with Nelder–Mead, 10 iterations) **failed** in our run: on Quokka it stopped at $\theta = -0.43$ with loss 0.89 instead of $\theta = \pi$ (the earlier simulator run stopped at $\theta = -0.47$ with loss 0.87). Near $\theta = 0$ the loss $\cos^4(\theta/2)$ is almost flat, and differences between nearby points are smaller than the shot noise, so the simplex method (Nelder & Mead, 1965) could not tell which way to move in 10 iterations. This small failure illustrates a real problem in quantum machine learning: noise plus flat loss landscapes make training hard, and in large circuits this becomes the 'barren plateau' problem (McClean et al., 2018). SPSA was more robust because it uses large, random perturbations.

6.9 What is our overall judgement for Task 4?

- **Input:** pixels → X gates (basis encoding) in `create_circuit`; **model:** 6 RY angles in a TTN of RY–RY–CNOT blocks; **output:** P(q3 = 1).
- **Monitoring:** on the Quokka device (quokka2) the loss fell from 0.497 to 0.000 in 100 iterations; time per iteration was flat at about 4.05 s.
- **Efficiency:** at 4 qubits, simulation without the device is about 3490 times faster; real devices only pay off where simulation is impossible.
- **Effectiveness:** the circuit solved an XOR-type problem that linear classical models could not. This comes from its CNOT structure, not from a proven quantum advantage (Benedetti et al., 2019).

7. Our Computer versus Quokka: Performance and Outcome

> Lecturer's requirement: compare the outcome and the performance of our local computer with the Quokka device, and write the comparison in the report. Scripts: Device_Comparison/compare_laptop_vs_quokka.py and scaling_laptop_vs_quokka.py (menu option C of run_project). Data: Device_Comparison/results/.

7.1 How did we compare them fairly?

We sent **exactly the same QASM text** to every target, so only the machine changes. The targets were our own computer (Qiskit Aer simulator, called "this computer" in the charts) and the four Quokka devices that were online (quokka2 to quokka5). Each target ran each circuit **five times with 1000 shots**, after one untimed warm-up run, which loads Qiskit locally and opens the network connection to Quokka. We measured three things:

- **Wall-clock time:** how long the user waits for 1000 shots, including the internet for Quokka.
- **Device time (`server_time_ms`):** the time Quokka itself reports for the work, without the network.
- **Outcome quality:** the total variation distance (TVD) between the measured distribution (5 × 1000 = 5000 shots) and the exact theory. 0 means perfect agreement.

The comparison was run on 4 November 2026 between 18:02 and 18:03 NZDT on our development computer, a Linux machine with Python 3 and Qiskit Aer 0.18. Because the lecturer asked for *our laptop*, the same comparison runs on any Windows or Mac laptop with one click (menu option **C**). The absolute numbers depend on the computer and the internet connection, but the pattern below does not.

7.2 Which is faster for our circuits?

Table 19. *Mean time per 1000-shot run (5 repeats) and distance from theory; Task 1 Bell state and Task 2 half adder*

Target	Bell: time (s)	Bell: device (ms)	Adder: time (s)	Adder: device (ms)	Bell TVD	Adder TVD
This computer (Aer)	0.078	–	0.073	–	0.002	0.007
quokka2	0.954	2.2	0.943	3.4	0.006	0.011
quokka3	0.896	2.9	1.158	3.3	0.008	0.009
quokka4	0.976	2.4	1.225	4.3	0.001	0.004
quokka5	1.057	2.5	0.937	3.3	0.004	0.015
quokka1 (and quokka6–11)	offline	–	offline	–	–	–

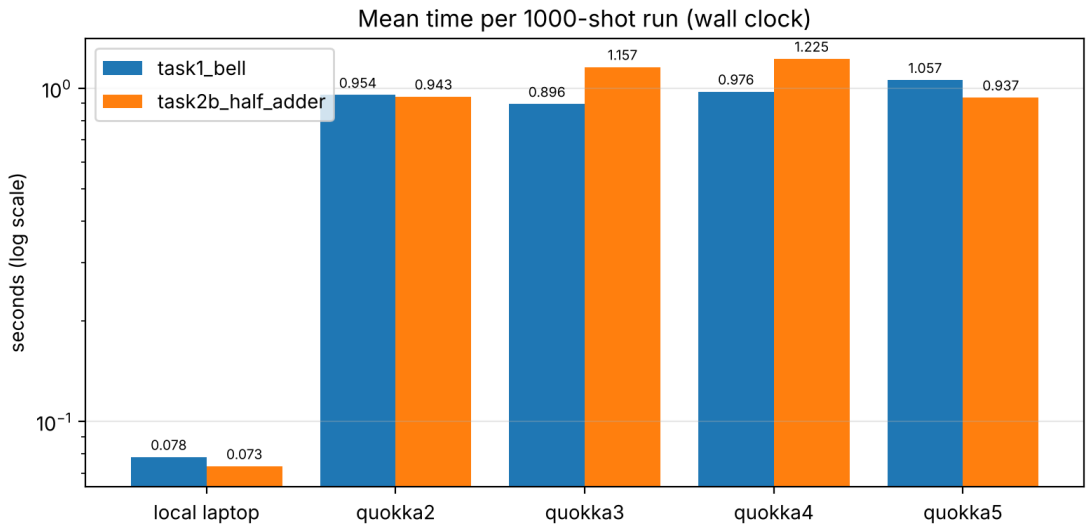


Figure 23. *Mean wall-clock time per 1000-shot run on this computer and on four Quokka devices (log scale)*

For small circuits **our computer is about 12–16 times faster** (0.07–0.08 s against 0.9–1.2 s). The reason is not that Quokka is slow: it reports only **2–4 ms** of work for 1000 shots. More than **99% of Quokka's time is the internet round trip** from our computer to the device and back, through the ngrok tunnel. Our computer has no network step, so it wins whenever the circuit is small enough to simulate in milliseconds.

7.3 Do they give the same outcome?

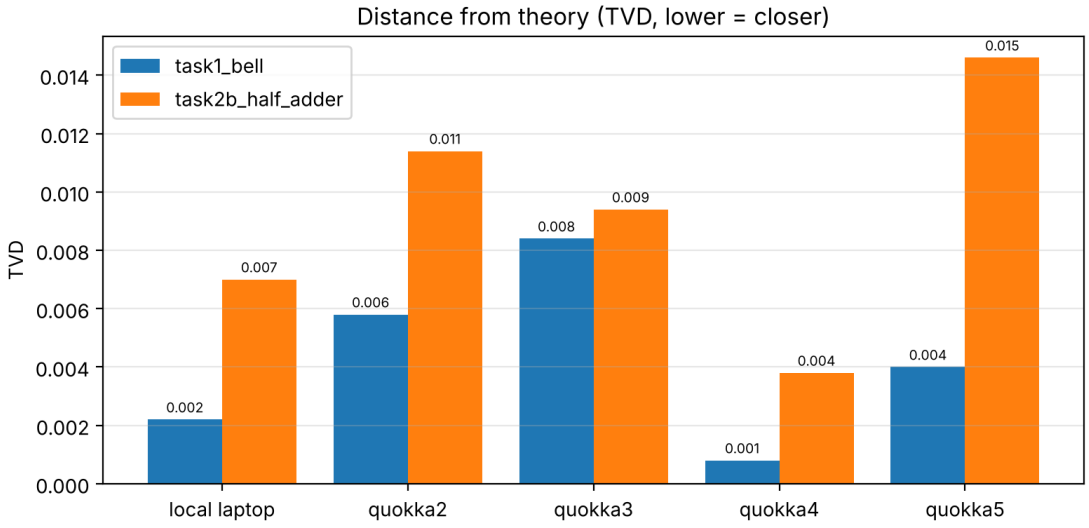


Figure 24. *Distance from theory (TVD) for this computer and four Quokka devices; all values are within shot noise*

Yes. Every target produced only the allowed outcomes: never `01/10` for the Bell state, and never a wrong addition for the half adder. All TVD values are **below 0.015**, which is the size of normal shot noise for 5000 shots. No device is "more accurate" than another: the small differences change from run to run. This is expected, because Quokka emulates an *ideal*, error-free quantum computer (UTS, 2024), and our simulator is also noise-free. A real superconducting or ion-trap computer would add a few percent of wrong outcomes (Preskill, 2018), and this comparison would then also measure hardware quality.

7.4 When would Quokka become the better choice?

A laptop stores the full quantum state: 2^n complex numbers of 16 bytes each. That is tiny for 2 or 4 qubits but doubles with every extra qubit. To find the cross-over point, we ran the same type of entangling circuit (H on every qubit, a CNOT chain and an RY layer) with **2 to 24 qubits** on our computer and on quokka2 (Figure 25).

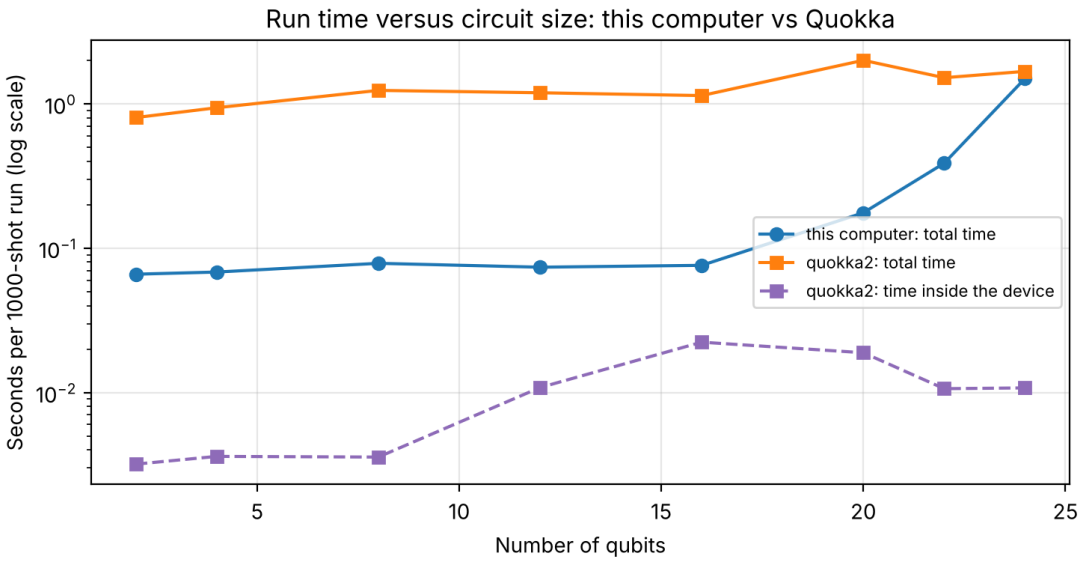


Figure 25. *Time per 1000-shot run versus number of qubits: this computer grows quickly, Quokka stays almost flat*

Table 20. *Scaling test (mean of 2 runs, 1000 shots each, 4 November 2026)*

Qubits	This computer (s)	quokka2 total (s)	quokka2 device (ms)	State size (MB)
2	0.066	0.80	3.2	0.00006
8	0.079	1.24	3.6	0.004
16	0.076	1.14	22.3	1.0
20	0.176	2.00	18.9	16.8
22	0.388	1.52	10.6	67.1
24	1.498	1.68	10.8	268.4

Up to about 16 qubits our computer's time hardly changes. From 20 qubits it roughly **quadruples for every two extra qubits** (0.18 → 0.39 → 1.50 s), following the 2^n size of the state. Quokka's total time stays between 0.8 and 2 s, and its device time stays near 10–20 ms. At **24 qubits the two are almost equal** (1.50 s against 1.68 s). If the trend continues, which is an extrapolation we did not test, our computer would need about 6 s at 26 qubits and about 25 s at 28 qubits. At 30 qubits it would need about 17 GB of memory for the state alone, more than a normal laptop has. Quokka is built for up to 30 qubits (UTS, 2024). So **for circuits above about 24 qubits, Quokka is the better choice**, and the network delay becomes small compared with the work.

7.5 What about a full workload such as Task 4?

Task 4 is a realistic workload: 100 training iterations with nine 1000-shot circuits each (6 timed requests plus 3 for monitoring). On the Quokka device (quokka2) the whole training took **606 s** for 900

circuits (4.05 s per iteration). The exact simulation of the same circuit on our computer (Local-A) took **0.18 s** and reached the same result. Before Quokka came back, the same shot-based run on the local simulator took 72 s (0.48 s per iteration). So Quokka made the training about 8 times slower than the local simulator and about 3,500 times slower than the exact local version. For this 4-qubit model, every one of the 900 requests pays the full network delay, and none of Quokka's strength is used.

7.6 What do we conclude?

- **Outcome:** identical within shot noise on every target, because both Quokka and our simulator are ideal (noise-free).
- **Performance for small circuits:** our computer is 12–16 times faster per run, because Quokka's time is almost all network delay (> 99%).
- **Performance for large circuits:** the laptop's cost doubles with every qubit, and Quokka's does not. The cross-over in our test is about 24 qubits, and above about 30 qubits a laptop runs out of memory.
- **Engineering lesson:** a remote quantum device should receive *batches* of large circuits, not many tiny ones. One request per circuit, as in Task 4, makes the network dominate.

8. Critical Discussion Across the Four Tasks

8.1 How does the evidence meet each learning outcome?

Table 21. *Evidence for each learning outcome*

LO	Requirement	Where this report shows it
LO1	Critically evaluate quantum protocols with the maths of quantum mechanics to validate performance claims	State evolution tables (Tables 5, 8); entanglement entropy; TVD and z-scores; the Z phase made visible by interference; the eigenstate explanation of NOT; the XOR analysis; the laptop-vs-Quokka and qubit-scaling measurements (Section 7)
LO2	Design and create quantum circuits for quantum algorithms to run on quantum computers	Cirq Bell circuit; Qiskit H–X + block C (CNOT, Z, CNOT, H) with checkpoint and control runs; original 4-qubit QASM half adder with automatic Toffoli decomposition; completed game gates; every circuit run on the Quokka device
LO4	Communicate application and critical analysis to software-industry audiences	WH-question structure, data-flow and pipeline diagrams, GitHub README with Colab instructions, Windows and macOS launchers, maintenance guide, labelled evidence files, dated screenshots and an honest account of the outage

8.2 What are the main limitations of our work?

- **Quokka availability.** The device was offline for several days, and only 4 of 11 cloud Quokkas were online when we re-ran the tasks. Our results depend on one device (quokka2) on one day; the comparison shows the other three online devices agree within shot noise.
- **An ideal device hides noise.** Quokka emulates a fault-tolerant machine, so our experiments cannot show decoherence or gate errors. A future step would be to add a Qiskit noise model, or a real noisy backend, and repeat Tasks 1 and 2 to see 01/10 leakage.
- **Small data in Task 4.** Four images and one test image are too few for strong conclusions about generalisation.
- **Mostly simulated gameplay evidence in Task 3.** Three games were played by clicking; the four detailed studies were replayed by script so they can be repeated exactly.
- **Timing depends on the network.** Quokka times include the internet round trip from our computer, so another location or connection would give different absolute numbers (the pattern stays the same).

8.3 What would we do next?

With more time we would: (1) send batches of circuits to Quokka in one request to reduce network time; (2) add an entangling CNOT move to the tic-tac-toe game so that tiles can be linked; (3) test the Task 4 classifier on larger images (3 × 3 = 9 qubits) to see when exact simulation slows down; and (4) add a noise model to measure how fidelity falls as circuits get deeper.

9. Development Process and Code Walkthrough

This section explains how the project was built, step by step, and what every part of the code does, so that a reader can follow, re-run and change it. Every code cell in the notebooks also carries inline comments.

9.1 What process did we follow?

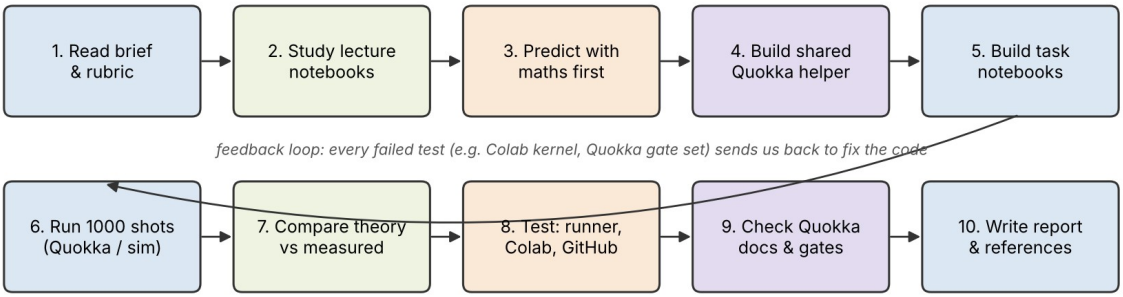


Figure 26. *Development process used for every task, with a feedback loop after each test*

1. **Read the brief and the rubric.** List what each task must show (e.g. "executed on the Quokka device", "Word document", "plots") and turn each item into a checklist.
2. **Study the lecture material.** The Week 4/7 `quokka_access` notebook gave the device address and request format; the Week 12 lab notebooks are the supplied code for Tasks 3 and 4.
3. **Predict with mathematics first.** Before running a circuit we wrote down its expected state and probabilities (state-evolution tables in Sections 3–4). This turns each run into a test.
4. **Build one shared helper** (`common/quokka_helper.py`) so that every task sends circuits the same way, with the same 1000 shots and the same evidence files.
5. **Build each task notebook** in the same order: objective → theory → circuit → code → run → theory-vs-measured table and chart → discussion.
6. **Run with 1000 shots** on Quokka, or on the labelled local simulator when Quokka is offline. The simulator uses a fixed seed, so every result can be repeated exactly.
7. **Compare theory with measurement** using the TVD and z-score, and explain every difference.
8. **Test the whole project end to end** with `run_project.py`, in Google Colab and from GitHub. Two real problems were found this way and fixed (Section 9.4).
9. **Check the device documentation.** The Quokka docs revealed its limited gate set, which led to automatic gate translation.
10. **Write the report** from the saved results, and verify every reference against Crossref and DataCite.

9.2 How does the shared helper work?

`common/quokka_helper.py` is the only file that talks to Quokka. Each notebook imports it as `qh` and normally calls just one function, `qh.run_qasm(qasm, shots=1000, label=..., save_dir=...)`.

Table 22. *Functions in common/quokka_helper.py*

Function	What it does	Why we need it
_quokka_devices() / device_url()	ordered device list from QUOKKA_URL, QUOKKA_NAME or quokka_config.json (quokka2, 3, 4, 5, 1)	change devices without editing code
clean_qasm(text)	removes include "qelib1.inc", comments, blank lines and barriers	Quokka has no qelib1 library and rejects extra lines
to_quokka_gates(text)	rewrites any gate outside {h, x, y, z, rx, ry, rz, cx, swap} using Qiskit's transpiler	Quokka's documented gate set has no s, t or ccx
_numeric_angles(text)	writes rz(pi/2) as rz(1.5707963…)	the docs only show numeric angles
run_on_device(text, shots, device)	POST {script, count} to one device with verify=False, as in the official notebooks; clear error if offline	the single place that contacts a device
run_on_quokka(text, shots)	tries each device in the list, starting with the one that worked last (failover)	one offline device no longer stops the project
run_locally(text, shots)	runs the same rewritten script on Qiskit Aer with a seed from a fixed sequence	repeatable, honest fallback
shots_to_counts / counts_to_probs	turns [[c0, c1], …] into {'c1c0': n} and probabilities	one bit order (Qiskit's) everywhere
total_variation_distance(p, q)	½ Σ |p − q|	one number for theory-vs-measurement agreement
run_qasm(…)	tries Quokka, falls back, times the run, saves JSON with source, device and server_time_ms	the function every task calls
quokka_status()	sends a 1-qubit test script to every device	used by run_project.py status
compare_devices(…)	runs one circuit on this computer and on every device (warm-up + repeats); time, device time, TVD	the laptop vs Quokka comparison (Section 7)
plot_comparison(series, …)	grouped bar chart with value labels	the same chart style in every task

```
def run_qasm(qasm_text, shots=1000, label="run", save_dir=None,
             allow_fallback=True):
    start = time.perf_counter()              # 1. start the stopwatch
    try:
        data = run_on_quokka(qasm_text, shots) # 2. Quokka first (translated)
        shot_list = list(data["result"].values())[0]
        source = SOURCE_QUOKKA
    except Exception as exc:                 # 3. offline -> same script on Aer
        shot_list = run_locally(qasm_text, shots)
        source = SOURCE_LOCAL
    counts = shots_to_counts(shot_list)      # 4. e.g. {'00': 508, '11': 492}
    ...                                      # 5. save JSON evidence
```

9.3 What does each notebook cell do?

Table 23. *Cell-by-cell guide to the four task notebooks*

Notebook	Cells (in order)	What the code does
All four	1. Title 2. Colab bootstrap	In Colab only: clone the GitHub repository, pip-install Cirq/Qiskit, move into the task folder. Skipped on a PC.
Task 1	3. Imports 4. Build circuit	LineQubit(0..1); H(q0); CNOT(q0, q1); measure into key 'c'.
	5. Theory 6. QASM	cirq.Simulator gives exact amplitudes (0.7071); cirq.qasm() exports OpenQASM 2.0; clean_qasm() prepares it.
	7. Run 8–10. Analysis	run_qasm with 1000 shots; results table with difference column; histogram; binomial z-score.
Task 2	A4. Build circuits	task2a_circuit(): h(0), x(1), then block C = cx, z, cx, h; three versions (checkpoint, full, control).
	A5. Entanglement test	Statevector after every gate; von Neumann entropy of q0 (0 → 1 → 1 → 0 bits).
	A6–A8. Run + compare	Three 1000-shot Quokka runs; theory dots vs measured bars; TVD; interpretation.
	B1–B2. Custom QASM	Load custom_half_adder.qasm (h, z, s, ccx, cx); draw; exact probabilities and phases.
	B3. Quokka gates + run	to_quokka_gates() rewrites s and ccx; checks probabilities are unchanged (True); 1000-shot run.
	B4. Truth table	Decodes each string into A, B, SUM, CARRY and checks every shot is a correct addition.
Task 3	Board class	9 qubits; H start; make_move adds x / ry(±π/4) / swap; measure 1 shot; countWinners with 8 lines.
	Game class	ipywidgets buttons and grid; handle_game routes clicks; unchanged rules.
	Recorded games	play_scripted() replays 4 games through the same Board code; 1000-shot study per game; heat map.
	Quokka export	removes reset, translates to h/ry/x/cx, sends Game 2 with 1000 shots.
Task 4	Data + split	generate_data(2) → 4 images; fixed-seed 75/25 split.
	Circuit	create_circuit: X gates encode pixels (input); 3 RY–RY–CNOT blocks in a tree; measure q3.
	SPSA + logging	Two loss evaluations per step; records output and time every iteration; 900 circuits.
	Local versions	Local-A NumPy state vector (checked against Qiskit); Local-B and Local-C classical models; comparison table and plots.

9.4 What went wrong during development, and how did we fix it?

Table 24. *Problems found by testing, and their fixes*

Problem	Cause	Fix
Quokka returned HTTP 404	device server offline (ERR_NGROK_3200)	labelled local fallback; status test; watcher; configurable address
Bell counts changed every run	unseeded local simulator	fixed seed sequence (LOCAL_SEED = 802) → repeatable evidence
Colab: No module named 'cirq'	nbconvert started a kernel from a different Python	run_project.py installs missing packages and registers its own kernel
Custom circuit would be rejected by Quokka	s and ccx are not in Quokka's QASM gate list	to_quokka_gates() rewrites them; equivalence checked
Tic-tac-toe could never have a winner	winners = () in supplied code	added the 8 winning lines
O button helped player X	comments swapped rotation directions	followed the game rules; corrected comments
Game failed outside Colab	google.colab.widgets only exists in Colab	standard ipywidgets layout
Clicks did nothing in Jupyter	callback output is discarded outside Colab	game screens drawn inside an Output widget; found by playing in Jupyter
Batch files failed on Windows	Unix line endings break batch labels	saved with CRLF; .gitattributes enforces it
Block C too small for the lecturer's intention	one CNOT read from the diagram	redesigned as CNOT, Z, CNOT, H with checkpoint and control runs
quokka1 still offline on 4 Nov	one fixed device address	device list with failover; status of every device; found quokka2–5 online
Task 4 used the simulator although Quokka was online	status text changed to 'online: quokka2, ...' but the check was == 'online'	check with startswith('online'); Task 4 re-run on Quokka

9.5 How did the code change over time? (Version history)

All work is version-controlled in the GitHub repository. Each commit message explains what changed and why, so the history is part of the documentation.

Table 25. *Commit history of github.com/anishnz/quantumcomputing_YB_Anish*

Commit	Date	What changed and why
5025e0c	31 Oct 2026 01:00	First version: four task notebooks, shared helper, runner, Colab notebook, alternative code, report and README.
7bb1d43	31 Oct 2026 01:07	Fix Colab run: the runner installs missing packages and uses its own Jupyter kernel.
47b49f6	31 Oct 2026 01:15	Report: Colab test-run evidence; README: Copy to Drive steps.
501d72d	31 Oct 2026 01:41	Use Quokka's documented gate set: circuits are translated automatically before sending.
c7eab4a	31 Oct 2026 02:10	Configurable Quokka address, connection diagnosis, verified references, this walkthrough, Task 3 and Task 4 Word documents, final ZIP.
ffc6b72	31 Oct 2026 02:37	Game fixed for Jupyter (Output widget); three games played by clicking, with screenshots.

Commit	Date	What changed and why
8e6a818	4 Nov 2026	Quokka device list with failover, device name and server time in every result, laptop vs Quokka comparison, dated screenshots.
(this version)	4 Nov 2026	Task 2 block C rebuilt after the lecturer's brief; all tasks re-run on Quokka (quokka2); qubit-scaling test; macOS launchers; report and reflections updated.

10. Conclusion

The project answered all four tasks with working, commented and repeatable code, and every task ran on the Quokka device. **Task 1** prepared the Bell state $|\Phi^+\rangle$ in Cirq and confirmed perfect correlation in 1000 shots on Quokka. **Task 2** followed the lecturer's brief. After the given H and X gates, our block C (CNOT, Z, CNOT, H) created an entangled pair, changed it with one phase gate, and identified it with certainty through a Bell measurement. Our original QASM half adder produced a correct addition in every one of 1000 shots. **Task 3** completed and debugged the quantum tic-tac-toe game, explained how each click becomes a gate, and used recorded and played games to show superposition, over-rotation, the eigenstate behaviour of NOT, and SWAP. **Task 4** explained the data encoding and the tree-shaped circuit of a variational classifier, logged and plotted all 100 iterations, and compared the circuit with three local versions. The local exact version was about 3490 times faster, but only circuit-shaped models solved the XOR-shaped data. The **laptop-versus-Quokka comparison** showed equal outcomes and explained the speed difference: network delay for small circuits, and the 2^n memory wall for large ones. The Quokka outage taught us to build with a labelled fallback and a device list, so that every result stays honest and repeatable. Across the project, the most important lesson was to **predict with mathematics first, then measure, then explain the difference**. That habit turned code that runs into evidence that convinces.

References

Bahrami, H. (2026a). *From quantum gates to quantum algorithms: Deutsch's algorithm* [Lecture slides, Week 7]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Bahrami, H. (2026b). *Lab 5 — Quantum tic-tac-toe* [Unpublished Jupyter notebook, Quantum_Tic_Tac_Toe__AS2.ipynb]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Bahrami, H. (2026c). *Lab 10 — Quantum ML* [Unpublished Jupyter notebook, Quantum_ML_AS2.ipynb]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Bahrami, H. (2026d). *Quokka access* [Unpublished Jupyter notebook, quokka_access.ipynb, Week 4]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Barenco, A., Bennett, C. H., Cleve, R., DiVincenzo, D. P., Margolus, N., Shor, P., Sleator, T., Smolin, J. A., & Weinfurter, H. (1995). Elementary gates for quantum computation. *Physical Review A*, *52*(5), 3457–3467. https://doi.org/10.1103/PhysRevA.52.3457

Bell, J. S. (1964). On the Einstein Podolsky Rosen paradox. *Physics Physique Fizika*, *1*(3), 195–200. https://doi.org/10.1103/PhysicsPhysiqueFizika.1.195

Benedetti, M., Lloyd, E., Sack, S., & Fiorentini, M. (2019). Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, *4*(4), Article 043001. https://doi.org/10.1088/2058-9565/ab4eb5

Bennett, C. H., & Wiesner, S. J. (1992). Communication via one- and two-particle operators on Einstein-Podolsky-Rosen states. *Physical Review Letters*, *69*(20), 2881–2884. https://doi.org/10.1103/PhysRevLett.69.2881

Cirq Developers. (2026). *Cirq* (Version 1.7.0) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.4062499

Cross, A. W., Bishop, L. S., Smolin, J. A., & Gambetta, J. M. (2017). *Open quantum assembly language* (arXiv:1707.03429). arXiv. https://doi.org/10.48550/arXiv.1707.03429

Einstein, A., Podolsky, B., & Rosen, N. (1935). Can quantum-mechanical description of physical reality be considered complete? *Physical Review*, *47*(10), 777–780. https://doi.org/10.1103/PhysRev.47.777

Goff, A. (2006). Quantum tic-tac-toe: A teaching metaphor for superposition in quantum mechanics. *American Journal of Physics*, *74*(11), 962–973. https://doi.org/10.1119/1.2213635

Grant, E., Benedetti, M., Cao, S., Hallam, A., Lockhart, J., Stojevic, V., Green, A. G., & Severini, S. (2018). Hierarchical quantum classifiers. *npj Quantum Information*, *4*, Article 65. https://doi.org/10.1038/s41534-018-0116-9

Hensen, B., Bernien, H., Dréau, A. E., Reiserer, A., Kalb, N., Blok, M. S., Ruitenberg, J., Vermeulen, R. F. L., Schouten, R. N., Abellán, C., Amaya, W., Pruneri, V., Mitchell, M. W., Markham, M., Twitchen, D. J., Elkouss, D., Wehner, S., Taminiau, T. H., & Hanson, R. (2015). Loophole-free Bell inequality violation using electron spins separated by 1.3 kilometres. *Nature*, *526*(7575), 682–686. https://doi.org/10.1038/nature15759

Huggins, W., Patil, P., Mitchell, B., Whaley, K. B., & Stoudenmire, E. M. (2019). Towards quantum machine learning with tensor networks. *Quantum Science and Technology*, *4*(2), Article 024001. https://doi.org/10.1088/2058-9565/aaea94

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., Johnson, B. R., & Gambetta, J. M. (2024). *Quantum computing with Qiskit* (arXiv:2405.08810). arXiv. https://doi.org/10.48550/arXiv.2405.08810

McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., & Neven, H. (2018). Barren plateaus in quantum neural network training landscapes. *Nature Communications*, *9*, Article 4812. https://doi.org/10.1038/s41467-018-07090-4

Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.

Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, *7*(4), 308–313. https://doi.org/10.1093/comjnl/7.4.308

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511976667

Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, *2*, 79. https://doi.org/10.22331/q-2018-08-06-79

Quantum logic gate. (n.d.). In *Wikipedia*. Retrieved October 31, 2026, from https://en.wikipedia.org/wiki/Quantum_logic_gate

Quokka Computing. (n.d.-a). *Quokka start here* [Google Colab notebook]. Retrieved November 4, 2026, from https://colab.research.google.com/drive/17sM33S5XXjiew5Ss9U00ygOqLNSUUMOP

Quokka Computing. (n.d.-b). *Quokka docs* [Google Colab notebook]. Retrieved October 31, 2026, from https://colab.research.google.com/drive/1hRRnMnF9ivumrtVbku2dMlHtrRULpIav

Schuld, M., & Petruccione, F. (2021). *Machine learning with quantum computers* (2nd ed.). Springer. https://doi.org/10.1007/978-3-030-83098-4

Spall, J. C. (1992). Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, *37*(3), 332–341. https://doi.org/10.1109/9.119632

University of Technology Sydney. (2024, June 25). *Quokka: A quantum hop into the future of computing*. https://www.uts.edu.au/news/2024/06/quokka-quantum-hop-future-computing

Vedral, V., Barenco, A., & Ekert, A. (1996). Quantum networks for elementary arithmetic operations. *Physical Review A*, *54*(1), 147–153. https://doi.org/10.1103/PhysRevA.54.147

Yanofsky, N. S., & Mannucci, M. A. (2008). *Quantum computing for computer scientists*. Cambridge University Press. https://doi.org/10.1017/CBO9780511813887

Appendix A – Submission Structure and How to Run

```
quantumcomputing_YB_Anish/            (GitHub repository)
├── README.md                         step-by-step run instructions
└── Assignment_2_Quantum/
    ├── Task_1_Bell_State/            notebook + results/
    ├── Task_2_Qiskit_QASM/           notebook, custom_half_adder.qasm, results/
    ├── Task_3_Quantum_TicTacToe/     completed + original notebook, results/
    ├── Task_4_Quantum_ML/            modified + original notebook, results/
    ├── Device_Comparison/            laptop vs Quokka scripts, results/, screenshots/
    ├── Alternative_Code/             second solution for each task + outputs/
    ├── common/quokka_helper.py       device list, failover, fallback, counts
    ├── Report/                       Assignment2.docx/.pdf, Reflections/
    ├── run_project.py                runner (menu, all, 1-4, alt, status, compare)
    ├── run_project.bat  run_all.bat  Windows: double-click
    ├── run_project.command  run_all.command   macOS: double-click
    ├── run_project.sh                Linux / macOS terminal
    ├── Colab_Run_All_Tasks.ipynb     whole project in Google Colab
    └── requirements.txt  MAINTENANCE.md  quokka_config.json
```

To reproduce every result: double-click `run_all.bat` (Windows) or `run_all.command` (macOS), run `python run_project.py all` (any computer) or open `Colab_Run_All_Tasks.ipynb` in Google Colab and choose Runtime → Run all. Each `results/` folder then holds the JSON (raw counts, source, time), CSV and PNG evidence used in this report, and `outputs_html/` holds a web-page copy of every notebook. Repository: https://github.com/anishnz/quantumcomputing_YB_Anish. `MAINTENANCE.md` explains common errors and safe changes.

Appendix B – Key Code Excerpts

B.1 Sending a circuit to Quokka with a labelled fallback (common/quokka_helper.py, shortened)

```
def run_qasm(qasm_text, shots=1000, label="run", save_dir=None,
             allow_fallback=True):
    try:
        data = run_on_quokka(qasm_text, shots)    # tries quokka2, 3, 4, 5, 1
        shot_list = list(data["result"].values())[0]   # [c0, c1] per shot
        source = SOURCE_QUOKKA
    except Exception as exc:                      # offline / error reply
        if not allow_fallback: raise
        shot_list = run_locally(qasm_text, shots) # SAME script on Aer
        source = SOURCE_LOCAL                     # never called Quokka data
    counts = shots_to_counts(shot_list)           # {'00': 501, '11': 499}
```

B.2 Completed moves in the tic-tac-toe game (Task 3, Board.make_move)

```
if self.function == 'Not':
    self.qc.x(cell)                    # [ADDED] bit flip: swap ownership
elif self.function == 'O':
    self.qc.ry(ROTATION, cell)         # [ADDED] quarter turn toward |1>
elif self.function == 'X':
    self.qc.ry(-ROTATION, cell)        # [ADDED] quarter turn toward |0>
elif (self.function == 'SWAP' and self.target >= 0
      and self.target != cell):        # [FIXED] needs a first tile
    self.qc.swap(cell, self.target)    # [ADDED] exchange two tiles
    self.target = -1                   # [FIXED] clear the first tile
```

B.3 Per-iteration monitoring added to SPSA (Task 4)

```
t0 = time.perf_counter()                            # [ADDED] start timer
grad = (func(xp) - func(xm)) / (2 * ck) * delta     # 2 evaluations
x = x - ak * grad                                   # update parameters
step_time = time.perf_counter() - t0                # [ADDED] iteration time
fx = func(x)                                        # [ADDED] output measure
history.append({"iteration": k + 1, "output": fx,
                "time_s": step_time})
```

Appendix C – Evidence Screenshots

These screenshots support the statements in the report. Screenshots of the Quokka website and devices carry a black banner with the exact capture time in New Zealand time. The original files are in `Device_Comparison/screenshots/` and `Task_3_Quantum_TicTacToe/results/gameplay/`. The two individual reflection reports are submitted as separate files in `Report/Reflections/`.

C.1 Quokka website and device status (4 November 2026)

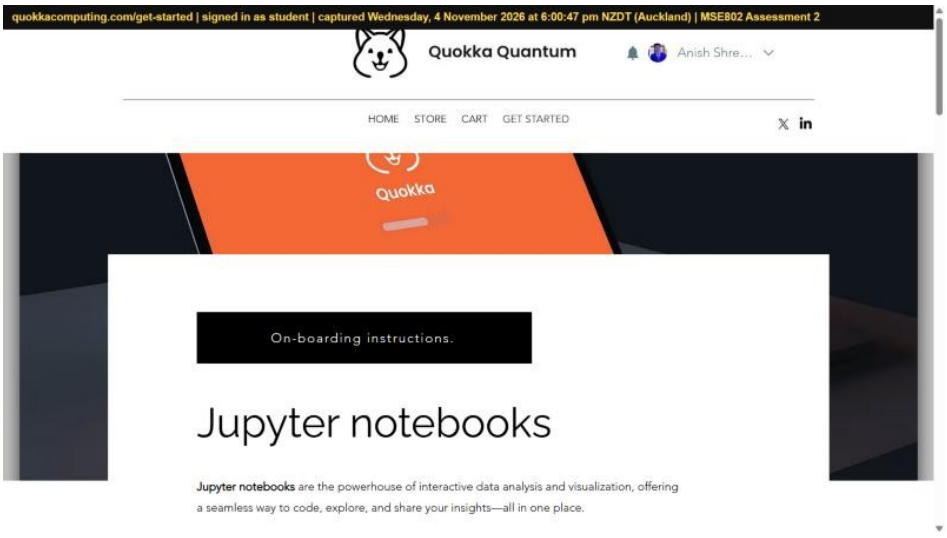


Figure 27. *Quokka website, Get Started page, signed in with the student account (captured 4 Nov 2026, 6:00 pm NZDT)*

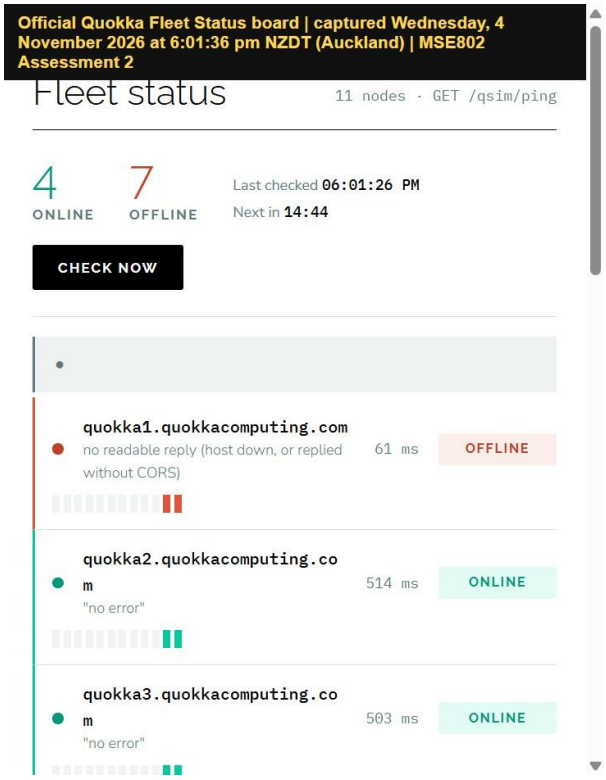


Figure 28. *Official Fleet Status board (part 1): 4 online, 7 offline; quokka1 offline, quokka2 and quokka3 online (6:01 pm NZDT)*

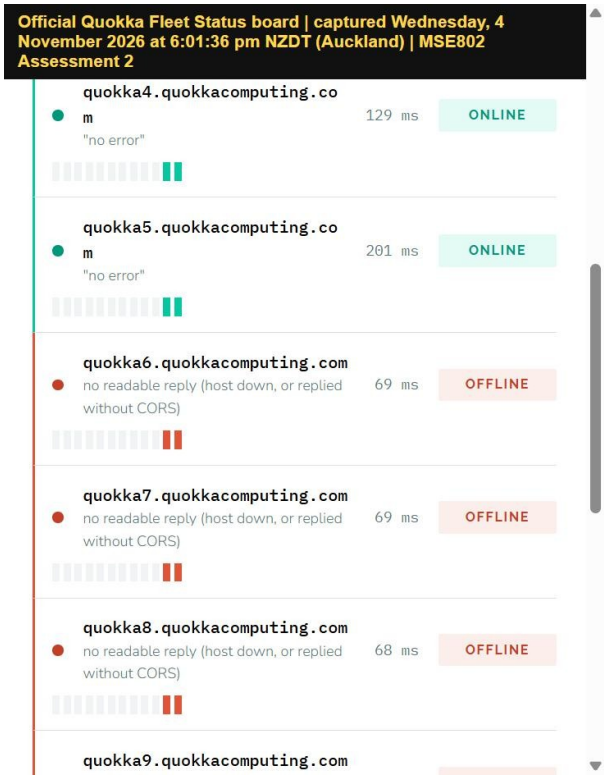


Figure 29. *Fleet Status board (part 2): quokka4 and quokka5 online; quokka6 to quokka9 offline*

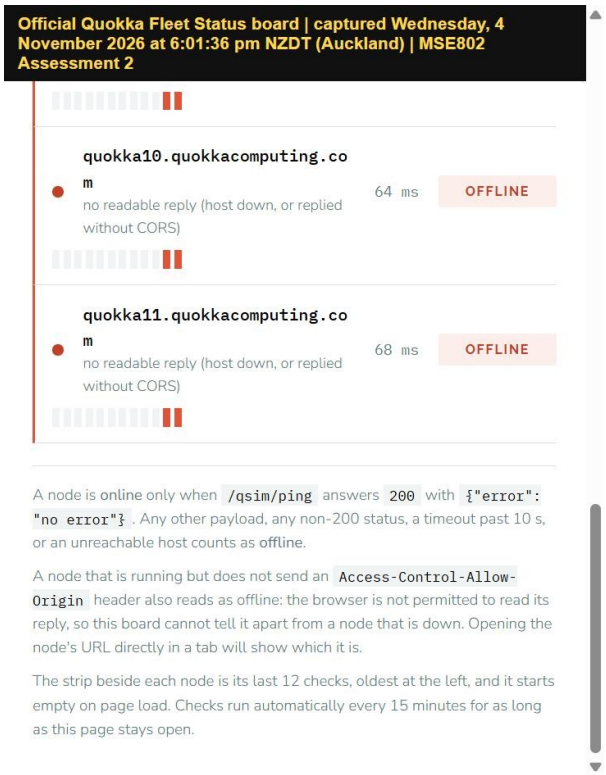


Figure 30. *Fleet Status board (part 3): quokka10 and quokka11 offline, and the board's own rule for 'online'*

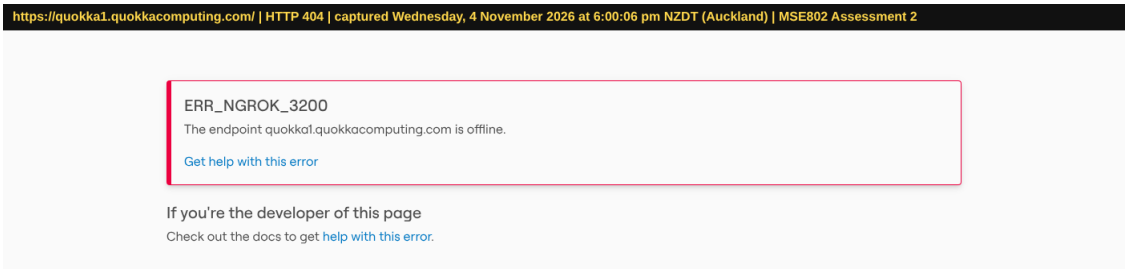


Figure 31. *quokka1 (the course notebook's address) opened in a browser: ERR_NGROK_3200, endpoint offline (5:59 pm NZDT)*



Figure 32. *quokka2 opened in a browser: the Quokka API answers (a GET request is not an operation; circuits are sent with POST /qsim/qasm)*

C.2 Task 3 games played by clicking in Jupyter Notebook

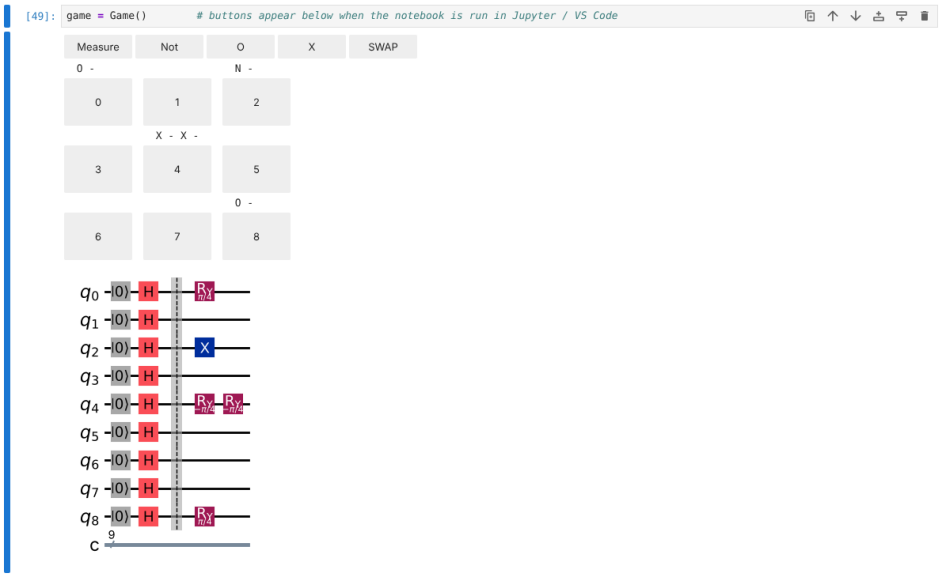


Figure 33. *Game A before Measure: moves X 4, X 4, O 0, O 8, Not 2 and the generated circuit*

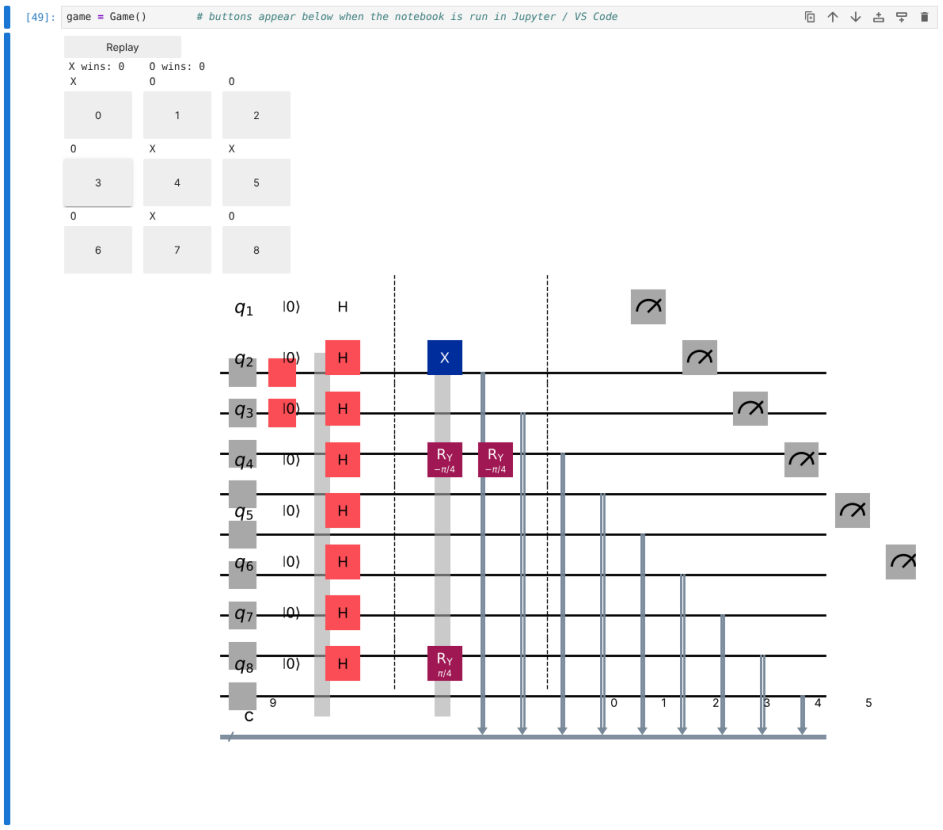


Figure 34. *Game A after Measure: one shot decided every tile (draw 0–0)*

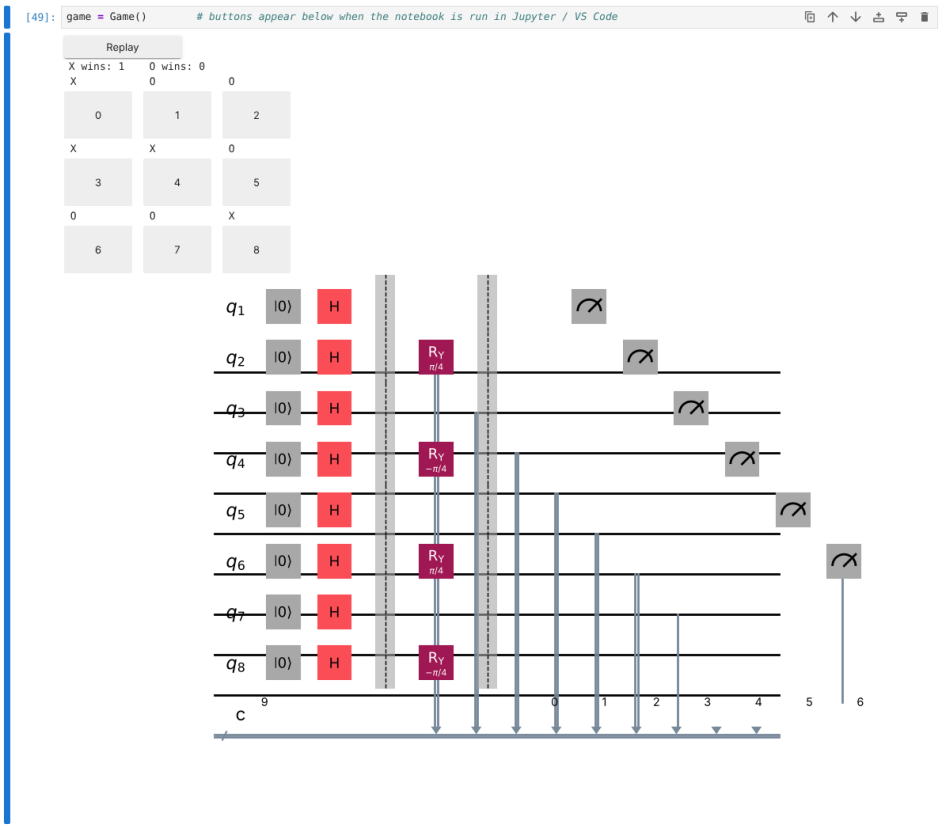


Figure 35. *Game B after Measure: X wins the main diagonal 0–4–8*

C.3 Notebooks executed on the Quokka device

```
Task1_Bell_State_Cirq_Quokka.ipynb - executed output (excerpt)

Quokka status: online: quokka2, quokka3, quokka4, quokka5 | offline: quokka1
Source of results : QUOKKA DEVICE
Counts            : {'00': 501, '11': 499}
  Outcome  Theory P  Counts (of 1000)  Measured P  Difference
```

Figure 36. *Task 1 notebook output: source QUOKKA DEVICE, device quokka2, 1000 shots*

```
Task2_Qiskit_Circuits_and_QASM.ipynb - executed output (excerpt)

Quokka status : online: quokka2, quokka3, quokka4, quokka5 | offline: quokka1
A-1 checkpoint (H,X,CNOT,Z)
   theory {'01': 0.5, '10': 0.5}
   counts {'01': 532, '10': 468}   [QUOKKA DEVICE - quokka2]
A-2 full circuit (H,X,CNOT,Z,CNOT,H)
   theory {'11': 1.0}
   counts {'11': 1000}   [QUOKKA DEVICE - quokka2]
A-3 control, no Z (H,X,CNOT,CNOT,H)
   theory {'10': 1.0}
   counts {'10': 1000}   [QUOKKA DEVICE - quokka2]
Run           A-1 checkpoint (H,X,CNOT,Z)  \
Run           A-2 full circuit (H,X,CNOT,Z,CNOT,H)  \
Run           A-3 control, no Z (H,X,CNOT,CNOT,H)
TVD A-1 checkpoint (H,X,CNOT,Z): 0.0320
TVD A-2 full circuit (H,X,CNOT,Z,CNOT,H): 0.0000
TVD A-3 control, no Z (H,X,CNOT,CNOT,H): 0.0000
Source : QUOKKA DEVICE
Counts : {'0000': 261, '0111': 242, '1001': 250, '1010': 247}
TVD (theory vs measured): 0.011
```

Figure 37. *Task 2 notebook output: the three Circuit 1 runs on quokka2 with their theory*

```
Task4_Quantum_ML_Modified.ipynb - executed output (excerpt)

Quokka status: online: quokka2, quokka3, quokka4, quokka5 | offline: quokka1
Quantum circuits will run on: QUOKKA DEVICE
Ran on: QUOKKA DEVICE
Total time 606.2 s, circuits executed: 900
                               Version  Total time (s)  \
0               Quantum circuit (Quokka)         606.193
```

Figure 38. *Task 4 notebook output: source of the training run and the comparison table*